

Our study of data structures thus far has focused primarily upon the efficiency of computations, as measured by the number of primitive operations that are executed on a central processing unit (CPU). In practice, the performance of a computer system is also greatly impacted by the management of the computer's memory systems. In our analysis of data structures, we have provided asymptotic bounds for the overall amount of memory used by a data structure. In this chapter, we consider more subtle issues involving the use of a computer's memory system.

We first discuss ways in which memory is allocated and deallocated during the execution of a computer program, and the impact that this has on the program's performance. Second, we discuss the complexity of multilevel memory hierarchies in today's computer systems. Although we often abstract a computer's memory as consisting of a single pool of interchangeable locations, in practice, the data used by an executing program is stored and transferred between a combination of physical memories (e.g., CPU registers, caches, internal memory, and external memory). We consider the use of classic data structures in the algorithms used to manage memory, and how the use of memory hierarchies impacts the choice of data structures and algorithms for classic problems such as searching and sorting.

15.1 Memory Management

In order to implement any data structure on an actual computer, we need to use computer memory. Computer memory is organized into a sequence of **words**, each of which typically consists of 4, 8, or 16 bytes (depending on the computer). These memory words are numbered from 0 to $N - 1$, where N is the number of memory words available to the computer. The number associated with each memory word is known as its **memory address**. Thus, the memory in a computer can be viewed as basically one giant array of memory words. For example, in Figure 5.1 of Section 5.2, we portrayed a section of the computer's memory as follows:



In order to run programs and store information, the computer's memory must be **managed** so as to determine what data is stored in what memory cells. In this section, we discuss the basics of memory management, most notably describing the way in which memory is allocated to store new objects, the way in which portions of memory are deallocated and reclaimed, when no longer needed, and the way in which the Python interpreter uses memory in completing its tasks.

15.1.1 Memory Allocation

With Python, all objects are stored in a pool of memory, known as the *memory heap* or *Python heap* (which should not be confused with the “heap” data structure presented in Chapter 9). When a command such as

```
w = Widget()
```

is executed, assuming `Widget` is the name of a class, a new instance of the class is created and stored somewhere within the memory heap. The Python interpreter is responsible for negotiating the use of space with the operating system and for managing the use of the memory heap when executing a Python program.

The storage available in the memory heap is divided into *blocks*, which are contiguous array-like “chunks” of memory that may be of variable or fixed sizes. The system must be implemented so that it can quickly allocate memory for new objects. One popular method is to keep contiguous “holes” of available free memory in a linked list, called the *free list*. The links joining these holes are stored inside the holes themselves, since their memory is not being used. As memory is allocated and deallocated, the collection of holes in the free lists changes, with the unused memory being separated into disjoint holes divided by blocks of used memory. This separation of unused memory into separate holes is known as *fragmentation*. The problem is that it becomes more difficult to find large continuous chunks of memory, when needed, even though an equivalent amount of memory may be unused (yet fragmented). Therefore, we would like to minimize fragmentation as much as possible.

There are two kinds of fragmentation that can occur. *Internal fragmentation* occurs when a portion of an allocated memory block is unused. For example, a program may request an array of size 1000, but only use the first 100 cells of this array. There is not much that a run-time environment can do to reduce internal fragmentation. *External fragmentation*, on the other hand, occurs when there is a significant amount of unused memory between several contiguous blocks of allocated memory. Since the run-time environment has control over where to allocate memory when it is requested, the run-time environment should allocate memory in a way to try to reduce external fragmentation as much as reasonably possible.

Several heuristics have been suggested for allocating memory from the heap so as to minimize external fragmentation. The *best-fit algorithm* searches the entire free list to find the hole whose size is closest to the amount of memory being requested. The *first-fit algorithm* searches from the beginning of the free list for the first hole that is large enough. The *next-fit algorithm* is similar, in that it also searches the free list for the first hole that is large enough, but it begins its search from where it left off previously, viewing the free list as a circularly linked list (Section 7.2). The *worst-fit algorithm* searches the free list to find the largest hole of available memory, which might be done faster than a search of the entire free list

if this list were maintained as a priority queue (Chapter 9). In each algorithm, the requested amount of memory is subtracted from the chosen memory hole and the leftover part of that hole is returned to the free list.

Although it might sound good at first, the best-fit algorithm tends to produce the worst external fragmentation, since the leftover parts of the chosen holes tend to be small. The first-fit algorithm is fast, but it tends to produce a lot of external fragmentation at the front of the free list, which slows down future searches. The next-fit algorithm spreads fragmentation more evenly throughout the memory heap, thus keeping search times low. This spreading also makes it more difficult to allocate large blocks, however. The worst-fit algorithm attempts to avoid this problem by keeping contiguous sections of free memory as large as possible.

15.1.2 Garbage Collection

In some languages, like C and C++, the memory space for objects must be explicitly deallocated by the programmer, which is a duty often overlooked by beginning programmers and is the source of frustrating programming errors even for experienced programmers. The designers of Python instead placed the burden of memory management entirely on the interpreter. The process of detecting “stale” objects, deallocating the space devoted to those objects, and returning the reclaimed space to the free list is known as *garbage collection*.

To perform automated garbage collection, there must first be a way to detect those objects that are no longer necessary. Since the interpreter cannot feasibly analyze the semantics of an arbitrary Python program, it relies on the following conservative rule for reclaiming objects. In order for a program to access an object, it must have a direct or indirect reference to that object. We will define such objects to be *live objects*. In defining a live object, a *direct reference* to an object is in the form of an identifier in an active namespace (i.e., the global namespace, or the local namespace for any active function). For example, immediately after the command `w = Widget()` is executed, identifier `w` will be defined in the current namespace as a reference to the new widget object. We refer to all such objects with direct references as *root objects*. An *indirect reference* to a live object is a reference that occurs within the state of some other live object. For example, if the widget instance in our earlier example maintains a list as an attribute, that list is also a live object (as it can be reached indirectly through use of identifier `w`). The set of live objects are defined recursively; thus, any objects that are referenced within the list that is referenced by the widget are also classified as live objects.

The Python interpreter assumes that live objects are the active objects currently being used by the running program; these objects should *not* be deallocated. Other objects can be garbage collected. Python relies on the following two strategies for determining which objects are live.

Reference Counts

Within the state of every Python object is an integer known as its *reference count*. This is the count of how many references to the object exist anywhere in the system. Every time a reference is assigned to this object, its reference count is incremented, and every time one of those references is reassigned to something else, the reference count for the former object is decremented. The maintenance of a reference count for each object adds $O(1)$ space per object, and the increments and decrements to the count add $O(1)$ additional computation time per such operations.

The Python interpreter allows a running program to examine an object's reference count. Within the `sys` module there is a function named `getrefcount` that returns an integer equal to the reference count for the object sent as a parameter. It is worth noting that because the formal parameter of that function is assigned to the actual parameter sent by the caller, there is temporarily one additional reference to that object in the local namespace of the function at the time the count is reported.

The advantage of having a reference count for each object is that if an object's count is ever decremented to zero, that object cannot possibly be a live object and therefore the system can immediately deallocate the object (or place it in a queue of objects that are ready to be deallocated).

Cycle Detection

Although it is clear that an object with a reference count of zero cannot be a live object, it is important to recognize that an object with a nonzero reference count need not qualify as live. There may exist a group of objects that have references to each other, even though none of those objects are reachable from a root object.

For example, a running Python program may have an identifier, `data`, that is a reference to a sequence implemented using a doubly linked list. In this case, the list referenced by `data` is a root object, the header and trailer nodes that are stored as attributes of the list are live objects, as are all the intermediate nodes of the list that are indirectly referenced and all the elements that are referenced as elements of those nodes. If the identifier, `data`, were to go out of scope, or to be reassigned to some other object, the reference count for the list instance may go to zero and be garbage collected, but the reference counts for all of the nodes would remain nonzero, stopping them from being garbage collected by the simple rule above.

Every so often, in particular when the available space in the memory heap is becoming scarce, the Python interpreter uses a more advanced form of garbage collection to reclaim objects that are unreachable, despite their nonzero reference counts. There are different algorithms for implementing cycle detection. (The mechanics of garbage collection in Python are abstracted in the `gc` module, and may vary depending on the implementation of the interpreter.) A classic algorithm for garbage collection is the *mark-sweep algorithm*, which we next discuss.

The Mark-Sweep Algorithm

In the mark-sweep garbage collection algorithm, we associate a “mark” bit with each object that identifies whether that object is live. When we determine at some point that garbage collection is needed, we suspend all other activity and clear the mark bits of all the objects currently allocated in the memory heap. We then trace through the active namespaces and we mark all the root objects as “live.” We must then determine all the other live objects—the ones that are reachable from the root objects. To do this efficiently, we can perform a depth-first search (see Section 14.3.1) on the directed graph that is defined by objects reference other objects. In this case, each object in the memory heap is viewed as a vertex in a directed graph, and the reference from one object to another is viewed as a directed edge. By performing a directed DFS from each root object, we can correctly identify and mark each live object. This process is known as the “mark” phase. Once this process has completed, we then scan through the memory heap and reclaim any space that is being used for an object that has not been marked. At this time, we can also optionally coalesce all the allocated space in the memory heap into a single block, thereby eliminating external fragmentation for the time being. This scanning and reclamation process is known as the “sweep” phase, and when it completes, we resume running the suspended program. Thus, the mark-sweep garbage collection algorithm will reclaim unused space in time proportional to the number of live objects and their references plus the size of the memory heap.

Performing DFS In-Place

The mark-sweep algorithm correctly reclaims unused space in the memory heap, but there is an important issue we must face during the mark phase. Since we are reclaiming memory space at a time when available memory is scarce, we must take care not to use extra space during the garbage collection itself. The trouble is that the DFS algorithm, in the recursive way we have described it in Section 14.3.1, can use space proportional to the number of vertices in the graph. In the case of garbage collection, the vertices in our graph are the objects in the memory heap; hence, we probably do not have this much memory to use. So our only alternative is to find a way to perform DFS in-place rather than recursively, that is, we must perform DFS using only a constant amount of additional storage.

The main idea for performing DFS in-place is to simulate the recursion stack using the edges of the graph (which in the case of garbage collection correspond to object references). When we traverse an edge from a visited vertex v to a new vertex w , we change the edge (v, w) stored in v 's adjacency list to point back to v 's parent in the DFS tree. When we return back to v (simulating the return from the “recursive” call at w), we can then switch the edge we modified to point back to w , assuming we have some way to identify which edge we need to change back.

15.1.3 Additional Memory Used by the Python Interpreter

We have discussed, in Section 15.1.1, how the Python interpreter allocates memory for objects within a memory heap. However, this is not the only memory that is used when executing a Python program. In this section, we discuss some other important uses of memory.

The Run-Time Call Stack

Stacks have a most important application to the run-time environment of Python programs. A running Python program has a private stack, known as the *call stack* or *Python interpreter stack*, that is used to keep track of the nested sequence of currently active (that is, nonterminated) invocations of functions. Each entry of the stack is a structure known as an *activation record* or *frame*, storing important information about an invocation of a function.

At the top of the call stack is the activation record of the *running call*, that is, the function activation that currently has control of the execution. The remaining elements of the stack are activation records of the *suspended calls*, that is, functions that have invoked another function and are currently waiting for that other function to return control when it terminates. The order of the elements in the stack corresponds to the chain of invocations of the currently active functions. When a new function is called, an activation record for that call is pushed onto the stack. When it terminates, its activation record is popped from the stack and the Python interpreter resumes the processing of the previously suspended call.

Each activation record includes a dictionary representing the local namespace for the function call. (See Sections 1.10 and 2.5 for further discussion of namespaces). The namespace maps identifiers, which serve as parameters and local variables, to object values, although the objects being referenced still reside in the memory heap. The activation record for a function call also includes a reference to the function definition itself, and a special variable, known as the *program counter*, to maintain the address of the statement within the function that is currently executing. When one function returns control to another, the stored program counter for the suspended function allows the interpreter to properly continue execution of that function.

Implementing Recursion

One of the benefits of using a stack to implement the nesting of function calls is that it allows programs to use *recursion*. That is, it allows a function to call itself, as discussed in Chapter 4. We implicitly described the concept of the call stack and the use of activation records within our portrayal of *recursion traces* in

that chapter. Interestingly, early programming languages, such as Cobol and Fortran, did not originally use call stacks to implement function calls. But because of the elegance and efficiency that recursion allows, almost all modern programming languages utilize a call stack for function calls, including the current versions of classic languages like Cobol and Fortran.

Each box of a recursive trace corresponds to an activation record that is placed on the call stack during the execution of a recursive function. At any point in time, the content of the call stack corresponds to the chain of boxes from the initial function invocation to the current one. To better illustrate how a call stack is used by recursive functions, we refer back to the Python implementation of the classic recursive definition of the factorial function,

$$n! = n(n-1)(n-2) \cdots 1,$$

with the code originally given in Code Fragment 4.1, and the recursive trace in Figure 4.1. The first time we call factorial, its activation record includes a namespace storing the parameter value n . The function recursively calls itself to compute $(n-1)!$, causing a new activation record, with its own namespace and parameter, to be pushed onto the call stack. In turn, this recursive invocation calls itself to compute $(n-2)!$, and so on. The chain of recursive invocations, and thus the call stack, grows up to size $n+1$, with the most deeply nested call being factorial(0), which returns 1 without any further recursion. The run-time stack allows several invocations of the factorial function to exist simultaneously. Each has an activation record that stores the value of its parameter, and eventually the value to be returned. When the first recursive call eventually terminates, it returns $(n-1)!$, which is then multiplied by n to compute $n!$ for the original call of the factorial method.

The Operand Stack

Interestingly, there is actually another place where the Python interpreter uses a stack. Arithmetic expressions, such as $((a+b) * (c+d))/e$, are evaluated by the interpreter using an *operand stack*. In Section 8.5 we described how to evaluate an arithmetic expression using a postorder traversal of an explicit expression tree. We described that algorithm in a recursive way; however, this recursive description can be simulated using a nonrecursive process that maintains an explicit operand stack. A simple binary operation, such as $a+b$, is computed by pushing a on the stack, pushing b on the stack, and then calling an instruction that pops the top two items from the stack, performs the binary operation on them, and pushes the result back onto the stack. Likewise, instructions for writing and reading elements to and from memory involve the use of pop and push methods for the operand stack.

15.2 Memory Hierarchies and Caching

With the increased use of computing in society, software applications must manage extremely large data sets. Such applications include the processing of online financial transactions, the organization and maintenance of databases, and analyses of customers' purchasing histories and preferences. The amount of data can be so large that the overall performance of algorithms and data structures sometimes depends more on the time to access the data than on the speed of the CPU.

15.2.1 Memory Systems

In order to accommodate large data sets, computers have a *hierarchy* of different kinds of memories, which vary in terms of their size and distance from the CPU. Closest to the CPU are the internal registers that the CPU itself uses. Access to such locations is very fast, but there are relatively few such locations. At the second level in the hierarchy are one or more memory *caches*. This memory is considerably larger than the register set of a CPU, but accessing it takes longer. At the third level in the hierarchy is the *internal memory*, which is also known as *main memory* or *core memory*. The internal memory is considerably larger than the cache memory, but also requires more time to access. Another level in the hierarchy is the *external memory*, which usually consists of disks, CD drives, DVD drives, and/or tapes. This memory is very large, but it is also very slow. Data stored through an external network can be viewed as yet another level in this hierarchy, with even greater storage capacity, but even slower access. Thus, the memory hierarchy for computers can be viewed as consisting of five or more levels, each of which is larger and slower than the previous level. (See Figure 15.1.) During the execution of a program, data is routinely copied from one level of the hierarchy to a neighboring level, and these transfers can become a computational bottleneck.

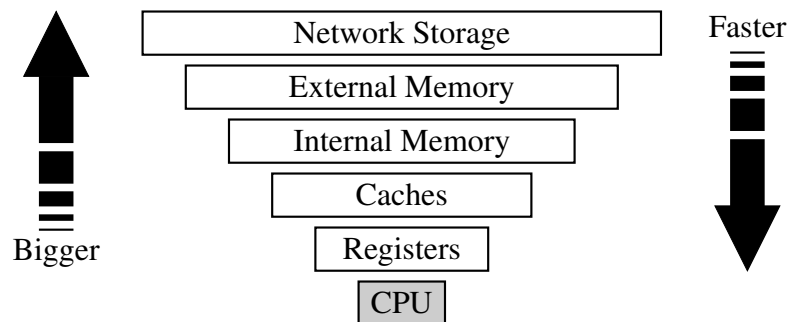


Figure 15.1: The memory hierarchy.

15.2.2 Caching Strategies

The significance of the memory hierarchy on the performance of a program depends greatly upon the size of the problem we are trying to solve and the physical characteristics of the computer system. Often, the bottleneck occurs between two levels of the memory hierarchy—the one that can hold all data items and the level just below that one. For a problem that can fit entirely in main memory, the two most important levels are the cache memory and the internal memory. Access times for internal memory can be as much as 10 to 100 times longer than those for cache memory. It is desirable, therefore, to be able to perform most memory accesses in cache memory. For a problem that does not fit entirely in main memory, on the other hand, the two most important levels are the internal memory and the external memory. Here the differences are even more dramatic, for access times for disks, the usual general-purpose external-memory device, are typically as much as 100000 to 1000000 times longer than those for internal memory.

To put this latter figure into perspective, imagine there is a student in Baltimore who wants to send a request-for-money message to his parents in Chicago. If the student sends his parents an email message, it can arrive at their home computer in about five seconds. Think of this mode of communication as corresponding to an internal-memory access by a CPU. A mode of communication corresponding to an external-memory access that is 500,000 times slower would be for the student to walk to Chicago and deliver his message in person, which would take about a month if he can average 20 miles per day. Thus, we should make as few accesses to external memory as possible.

Most algorithms are not designed with the memory hierarchy in mind, in spite of the great variance between access times for the different levels. Indeed, all of the algorithm analyses described in this book so far have assumed that all memory accesses are equal. This assumption might seem, at first, to be a great oversight—and one we are only addressing now in the final chapter—but there are good reasons why it is actually a reasonable assumption to make.

One justification for this assumption is that it is often necessary to assume that all memory accesses take the same amount of time, since specific device-dependent information about memory sizes is often hard to come by. In fact, information about memory size may be difficult to get. For example, a Python program that is designed to run on many different computer platforms cannot easily be defined in terms of a specific computer architecture configuration. We can certainly use architecture-specific information, if we have it (and we will show how to exploit such information later in this chapter). But once we have optimized our software for a certain architecture configuration, our software will no longer be device-independent. Fortunately, such optimizations are not always necessary, primarily because of the second justification for the equal-time memory-access assumption.

Caching and Blocking

Another justification for the memory-access equality assumption is that operating system designers have developed general mechanisms that allow most memory accesses to be fast. These mechanisms are based on two important *locality-of-reference* properties that most software possesses:

- **Temporal locality:** If a program accesses a certain memory location, then there is increased likelihood that it accesses that same location again in the near future. For example, it is common to use the value of a counter variable in several different expressions, including one to increment the counter's value. In fact, a common adage among computer architects is that a program spends 90 percent of its time in 10 percent of its code.
- **Spatial locality:** If a program accesses a certain memory location, then there is increased likelihood that it soon accesses other locations that are near this one. For example, a program using an array may be likely to access the locations of this array in a sequential or near-sequential manner.

Computer scientists and engineers have performed extensive software profiling experiments to justify the claim that most software possesses both of these kinds of locality of reference. For example, a nested for loop used to repeatedly scan through an array will exhibit both kinds of locality.

Temporal and spatial localities have, in turn, given rise to two fundamental design choices for multilevel computer memory systems (which are present in the interface between cache memory and internal memory, and also in the interface between internal memory and external memory).

The first design choice is called *virtual memory*. This concept consists of providing an address space as large as the capacity of the secondary-level memory, and of transferring data located in the secondary level into the primary level, when they are addressed. Virtual memory does not limit the programmer to the constraint of the internal memory size. The concept of bringing data into primary memory is called *caching*, and it is motivated by temporal locality. By bringing data into primary memory, we are hoping that it will be accessed again soon, and we will be able to respond quickly to all the requests for this data that come in the near future.

The second design choice is motivated by spatial locality. Specifically, if data stored at a secondary-level memory location ℓ is accessed, then we bring into primary-level memory a large block of contiguous locations that include the location ℓ . (See Figure 15.2.) This concept is known as *blocking*, and it is motivated by the expectation that other secondary-level memory locations close to ℓ will soon be accessed. In the interface between cache memory and internal memory, such blocks are often called *cache lines*, and in the interface between internal memory and external memory, such blocks are often called *pages*.

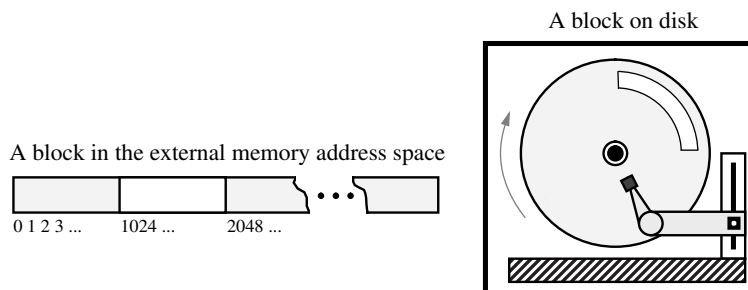


Figure 15.2: Blocks in external memory.

When implemented with caching and blocking, virtual memory often allows us to perceive secondary-level memory as being faster than it really is. There is still a problem, however. Primary-level memory is much smaller than secondary-level memory. Moreover, because memory systems use blocking, any program of substance will likely reach a point where it requests data from secondary-level memory, but the primary memory is already full of blocks. In order to fulfill the request and maintain our use of caching and blocking, we must remove some block from primary memory to make room for a new block from secondary memory in this case. Deciding which block to evict brings up a number of interesting data structure and algorithm design issues.

Caching in Web Browsers

For motivation, we consider a related problem that arises when revisiting information presented in Web pages. To exploit temporal locality of reference, it is often advantageous to store copies of Web pages in a *cache* memory, so these pages can be quickly retrieved when requested again. This effectively creates a two-level memory hierarchy, with the cache serving as the smaller, quicker internal memory, and the network being the external memory. In particular, suppose we have a cache memory that has m “slots” that can contain Web pages. We assume that a Web page can be placed in any slot of the cache. This is known as a *fully associative* cache.

As a browser executes, it requests different Web pages. Each time the browser requests such a Web page p , the browser determines (using a quick test) if p is unchanged and currently contained in the cache. If p is contained in the cache, then the browser satisfies the request using the cached copy. If p is not in the cache, however, the page for p is requested over the Internet and transferred into the cache. If one of the m slots in the cache is available, then the browser assigns p to one of the empty slots. But if all the m cells of the cache are occupied, then the computer must determine which previously viewed Web page to evict before bringing in p to take its place. There are, of course, many different policies that can be used to determine the page to evict.

Page Replacement Algorithms

Some of the better-known page replacement policies include the following (see Figure 15.3):

- **First-in, first-out (FIFO):** Evict the page that has been in the cache the longest, that is, the page that was transferred to the cache furthest in the past.
- **Least recently used (LRU):** Evict the page whose last request occurred furthest in the past.

In addition, we can consider a simple and purely random strategy:

- **Random:** Choose a page at random to evict from the cache.

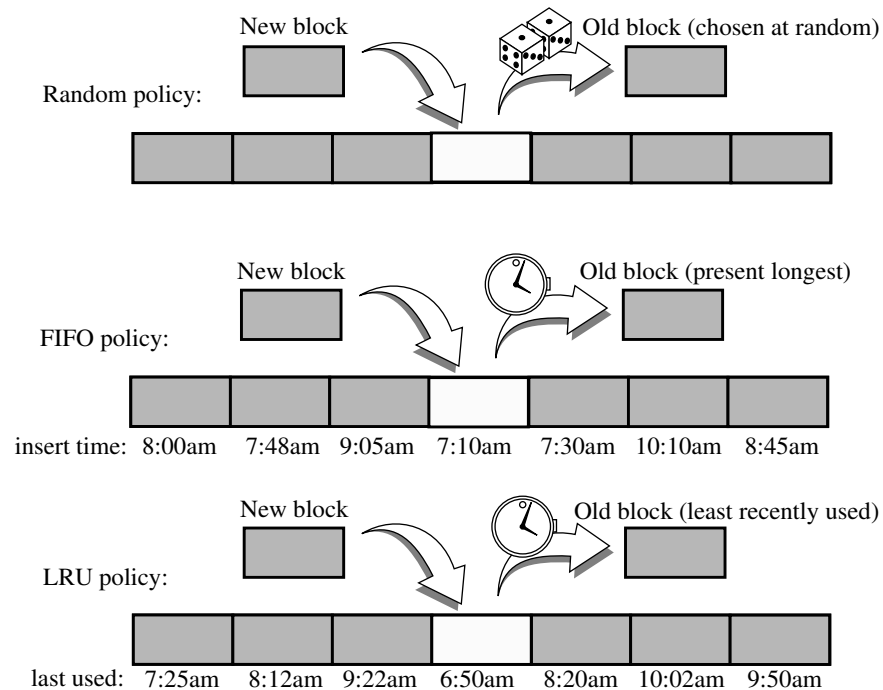


Figure 15.3: The random, FIFO, and LRU page replacement policies.

The random strategy is one of the easiest policies to implement, for it only requires a random or pseudo-random number generator. The overhead involved in implementing this policy is an $O(1)$ additional amount of work per page replacement. Moreover, there is no additional overhead for each page request, other than to determine whether a page request is in the cache or not. Still, this policy makes no attempt to take advantage of any temporal locality exhibited by a user's browsing.

The FIFO strategy is quite simple to implement, as it only requires a queue Q to store references to the pages in the cache. Pages are enqueued in Q when they are referenced by a browser, and then are brought into the cache. When a page needs to be evicted, the computer simply performs a dequeue operation on Q to determine which page to evict. Thus, this policy also requires $O(1)$ additional work per page replacement. Also, the FIFO policy incurs no additional overhead for page requests. Moreover, it tries to take some advantage of temporal locality.

The LRU strategy goes a step further than the FIFO strategy, for the LRU strategy explicitly takes advantage of temporal locality as much as possible, by always evicting the page that was least-recently used. From a policy point of view, this is an excellent approach, but it is costly from an implementation point of view. That is, its way of optimizing temporal and spatial locality is fairly costly. Implementing the LRU strategy requires the use of an adaptable priority queue Q that supports updating the priority of existing pages. If Q is implemented with a sorted sequence based on a linked list, then the overhead for each page request and page replacement is $O(1)$. When we insert a page in Q or update its key, the page is assigned the highest key in Q and is placed at the end of the list, which can also be done in $O(1)$ time. Even though the LRU strategy has constant-time overhead, using the implementation above, the constant factors involved, in terms of the additional time overhead and the extra space for the priority queue Q , make this policy less attractive from a practical point of view.

Since these different page replacement policies have different trade-offs between implementation difficulty and the degree to which they seem to take advantage of localities, it is natural for us to ask for some kind of comparative analysis of these methods to see which one, if any, is the best.

From a worst-case point of view, the FIFO and LRU strategies have fairly unattractive competitive behavior. For example, suppose we have a cache containing m pages, and consider the FIFO and LRU methods for performing page replacement for a program that has a loop that repeatedly requests $m + 1$ pages in a cyclic order. Both the FIFO and LRU policies perform badly on such a sequence of page requests, because they perform a page replacement on every page request. Thus, from a worst-case point of view, these policies are almost the worst we can imagine—they require a page replacement on every page request.

This worst-case analysis is a little too pessimistic, however, for it focuses on each protocol's behavior for one bad sequence of page requests. An ideal analysis would be to compare these methods over all possible page-request sequences. Of course, this is impossible to do exhaustively, but there have been a great number of experimental simulations done on page-request sequences derived from real programs. Based on these experimental comparisons, the LRU strategy has been shown to be usually superior to the FIFO strategy, which is usually better than the random strategy.

15.3 External Searching and B-Trees

Consider the problem of maintaining a large collection of items that does not fit in main memory, such as a typical database. In this context, we refer to the secondary-memory blocks as *disk blocks*. Likewise, we refer to the transfer of a block between secondary memory and primary memory as a *disk transfer*. Recalling the great time difference that exists between main memory accesses and disk accesses, the main goal of maintaining such a collection in external memory is to minimize the number of disk transfers needed to perform a query or update. We refer to this count as the *I/O complexity* of the algorithm involved.

Some Inefficient External-Memory Representations

A typical operation we would like to support is the search for a key in a map. If we were to store n items unordered in a doubly linked list, searching for a particular key within the list requires n transfers in the worst case, since each link hop we perform on the linked list might access a different block of memory.

We can reduce the number of block transfers by using an array-based sequence. A sequential search of an array can be performed using only $O(n/B)$ block transfers because of spatial locality of reference, where B denotes the number of elements that fit into a block. This is because the block transfer when accessing the first element of the array actually retrieves the first B elements, and so on with each successive block. It is worth noting that the bound of $O(n/B)$ transfers is only achieved when using a *compact array representation* (see Section 5.2.2). The standard Python list class is a referential container, and so even though the sequence of references are stored in an array, the actual elements that must be examined during a search are not generally stored sequentially in memory, resulting in n transfers in the worst case.

We could alternately store a sequence using a *sorted* array. In this case, a search performs $O(\log_2 n)$ transfers, via binary search, which is a nice improvement. But we do not get significant benefit from block transfers because each query during a binary search is likely in a different block of the sequence. As usual, update operations are expensive for a sorted array.

Since these simple implementations are I/O inefficient, we should consider the logarithmic-time internal-memory strategies that use balanced binary trees (for example, AVL trees or red-black trees) or other search structures with logarithmic average-case query and update times (for example, skip lists or splay trees). Typically, each node accessed for a query or update in one of these structures will be in a different block. Thus, these methods all require $O(\log_2 n)$ transfers in the worst case to perform a query or update operation. But we can do better! We can perform map queries and updates using only $O(\log_B n) = O(\log n / \log B)$ transfers.

15.3.1 (a, b) Trees

To reduce the number of external-memory accesses when searching, we can represent our map using a multiway search tree (Section 11.5.1). This approach gives rise to a generalization of the $(2, 4)$ tree data structure known as the (a, b) tree.

An (a, b) tree is a multiway search tree such that each node has between a and b children and stores between $a - 1$ and $b - 1$ entries. The algorithms for searching, inserting, and removing entries in an (a, b) tree are straightforward generalizations of the corresponding ones for $(2, 4)$ trees. The advantage of generalizing $(2, 4)$ trees to (a, b) trees is that a generalized class of trees provides a flexible search structure, where the size of the nodes and the running time of the various map operations depends on the parameters a and b . By setting the parameters a and b appropriately with respect to the size of disk blocks, we can derive a data structure that achieves good external-memory performance.

Definition of an (a, b) Tree

An (a, b) *tree*, where parameters a and b are integers such that $2 \leq a \leq (b + 1)/2$, is a multiway search tree T with the following additional restrictions:

Size Property: Each internal node has at least a children, unless it is the root, and has at most b children.

Depth Property: All the external nodes have the same depth.

Proposition 15.1: The height of an (a, b) tree storing n entries is $\Omega(\log n / \log b)$ and $O(\log n / \log a)$.

Justification: Let T be an (a, b) tree storing n entries, and let h be the height of T . We justify the proposition by establishing the following bounds on h :

$$\frac{1}{\log b} \log(n + 1) \leq h \leq \frac{1}{\log a} \log \frac{n + 1}{2} + 1.$$

By the size and depth properties, the number n'' of external nodes of T is at least $2a^{h-1}$ and at most b^h . By Proposition 11.7, $n'' = n + 1$. Thus,

$$2a^{h-1} \leq n + 1 \leq b^h.$$

Taking the logarithm in base 2 of each term, we get

$$(h - 1) \log a + 1 \leq \log(n + 1) \leq h \log b.$$

An algebraic manipulation of these inequalities completes the justification. ■

Search and Update Operations

We recall that in a multiway search tree T , each node v of T holds a secondary structure $M(v)$, which is itself a map (Section 11.5.1). If T is an (a, b) tree, then $M(v)$ stores at most b entries. Let $f(b)$ denote the time for performing a search in a map, $M(v)$. The search algorithm in an (a, b) tree is exactly like the one for multiway search trees given in Section 11.5.1. Hence, searching in an (a, b) tree T with n entries takes $O(\frac{f(b)}{\log a} \log n)$ time. Note that if b is considered a constant (and thus a is also), then the search time is $O(\log n)$.

The main application of (a, b) trees is for maps stored in external memory. Namely, to minimize disk accesses, we select the parameters a and b so that each tree node occupies a single disk block (so that $f(b) = 1$ if we wish to simply count block transfers). Providing the right a and b values in this context gives rise to a data structure known as the B-tree, which we will describe shortly. Before we describe this structure, however, let us discuss how insertions and removals are handled in (a, b) trees.

The insertion algorithm for an (a, b) tree is similar to that for a $(2, 4)$ tree. An overflow occurs when an entry is inserted into a b -node w , which becomes an illegal $(b + 1)$ -node. (Recall that a node in a multiway tree is a d -node if it has d children.) To remedy an overflow, we split node w by moving the median entry of w into the parent of w and replacing w with a $\lceil (b + 1)/2 \rceil$ -node w' and a $\lfloor (b + 1)/2 \rfloor$ -node w'' . We can now see the reason for requiring $a \leq (b + 1)/2$ in the definition of an (a, b) tree. Note that as a consequence of the split, we need to build the secondary structures $M(w')$ and $M(w'')$.

Removing an entry from an (a, b) tree is similar to what was done for $(2, 4)$ trees. An underflow occurs when a key is removed from an a -node w , distinct from the root, which causes w to become an illegal $(a - 1)$ -node. To remedy an underflow, we perform a transfer with a sibling of w that is not an a -node or we perform a fusion of w with a sibling that is an a -node. The new node w' resulting from the fusion is a $(2a - 1)$ -node, which is another reason for requiring $a \leq (b + 1)/2$.

Table 15.1 shows the performance of a map realized with an (a, b) tree.

Operation	Running Time
$M[k]$	$O\left(\frac{f(b)}{\log a} \log n\right)$
$M[k] = v$	$O\left(\frac{g(b)}{\log a} \log n\right)$
del $M[k]$	$O\left(\frac{g(b)}{\log a} \log n\right)$

Table 15.1: Time bounds for an n -entry map realized by an (a, b) tree T . We assume the secondary structure of the nodes of T support search in $f(b)$ time, and split and fusion operations in $g(b)$ time, for some functions $f(b)$ and $g(b)$, which can be made to be $O(1)$ when we are only counting disk transfers.

15.3.2 B-Trees

A version of the (a, b) tree data structure, which is the best-known method for maintaining a map in external memory, is called the “B-tree.” (See Figure 15.4.) A **B-tree of order d** is an (a, b) tree with $a = \lceil d/2 \rceil$ and $b = d$. Since we discussed the standard map query and update methods for (a, b) trees above, we restrict our discussion here to the I/O complexity of B-trees.

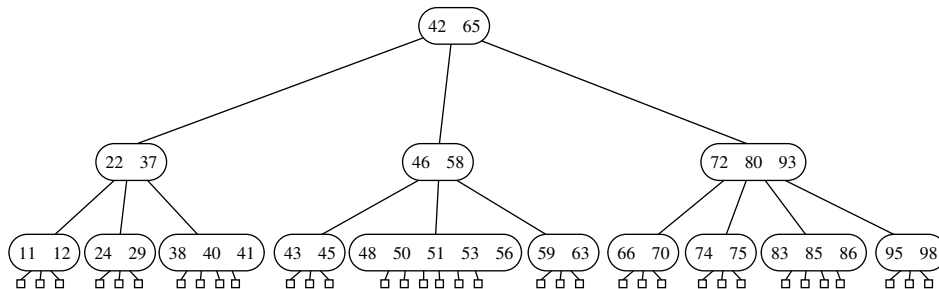


Figure 15.4: A B-tree of order 6.

An important property of B-trees is that we can choose d so that the d children references and the $d - 1$ keys stored at a node can fit compactly into a single disk block, implying that d is proportional to B . This choice allows us to assume that a and b are also proportional to B in the analysis of the search and update operations on (a, b) trees. Thus, $f(b)$ and $g(b)$ are both $O(1)$, for each time we access a node to perform a search or an update operation, we need only perform a single disk transfer.

As we have already observed above, each search or update requires that we examine at most $O(1)$ nodes for each level of the tree. Therefore, any map search or update operation on a B-tree requires only $O(\log_{\lceil d/2 \rceil} n)$, that is, $O(\log n / \log B)$, disk transfers. For example, an insert operation proceeds down the B-tree to locate the node in which to insert the new entry. If the node would **overflow** (to have $d + 1$ children) because of this addition, then this node is **split** into two nodes that have $\lfloor (d + 1)/2 \rfloor$ and $\lceil (d + 1)/2 \rceil$ children, respectively. This process is then repeated at the next level up, and will continue for at most $O(\log_B n)$ levels.

Likewise, if a remove operation results in a node **underflow** (to have $\lceil d/2 \rceil - 1$ children), then we move references from a sibling node with at least $\lceil d/2 \rceil + 1$ children or we perform a **fusion** operation of this node with its sibling (and repeat this computation at the parent). As with the insert operation, this will continue up the B-tree for at most $O(\log_B n)$ levels. The requirement that each internal node have at least $\lceil d/2 \rceil$ children implies that each disk block used to support a B-tree is at least half full. Thus, we have the following:

Proposition 15.2: A B-tree with n entries has I/O complexity $O(\log_B n)$ for search or update operation, and uses $O(n/B)$ blocks, where B is the size of a block.

15.4 External-Memory Sorting

In addition to data structures, such as maps, that need to be implemented in external memory, there are many algorithms that must also operate on input sets that are too large to fit entirely into internal memory. In this case, the objective is to solve the algorithmic problem using as few block transfers as possible. The most classic domain for such external-memory algorithms is the sorting problem.

Multiway Merge-Sort

An efficient way to sort a set S of n objects in external memory amounts to a simple external-memory variation on the familiar merge-sort algorithm. The main idea behind this variation is to merge many recursively sorted lists at a time, thereby reducing the number of levels of recursion. Specifically, a high-level description of this **multiway merge-sort** method is to divide S into d subsets $S_1, S_2, \dots, S_d$ of roughly equal size, recursively sort each subset S_i , and then simultaneously merge all d sorted lists into a sorted representation of S . If we can perform the merge process using only $O(n/B)$ disk transfers, then, for large enough values of n , the total number of transfers performed by this algorithm satisfies the following recurrence:

$$t(n) = d \cdot t(n/d) + cn/B,$$

for some constant $c \geq 1$. We can stop the recursion when $n \leq B$, since we can perform a single block transfer at this point, getting all of the objects into internal memory, and then sort the set with an efficient internal-memory algorithm. Thus, the stopping criterion for $t(n)$ is

$$t(n) = 1 \quad \text{if } n/B \leq 1.$$

This implies a closed-form solution that $t(n)$ is $O((n/B) \log_d(n/B))$, which is

$$O((n/B) \log(n/B) / \log d).$$

Thus, if we can choose d to be $\Theta(M/B)$, where M is the size of the internal memory, then the worst-case number of block transfers performed by this multiway merge-sort algorithm will be quite low. For reasons given in the next section, we choose

$$d = (M/B) - 1.$$

The only aspect of this algorithm left to specify, then, is how to perform the d -way merge using only $O(n/B)$ block transfers.

15.4.1 Multiway Merging

In a standard merge-sort (Section 12.2), the merge process combines two sorted sequences into one by repeatedly taking the smaller of the items at the front of the two respective lists. In a d -way merge, we repeatedly find the smallest among the items at the front of the d sequences and place it as the next element of the merged sequence. We continue until all elements are included.

In the context of an external-memory sorting algorithm, if main memory has size M and each block has size B , we can store up to M/B blocks within main memory at any given time. We specifically choose $d = (M/B) - 1$ so that we can afford to keep one block from each input sequence in main memory at any given time, and to have one additional block to use as a buffer for the merged sequence. (See Figure 15.5.)

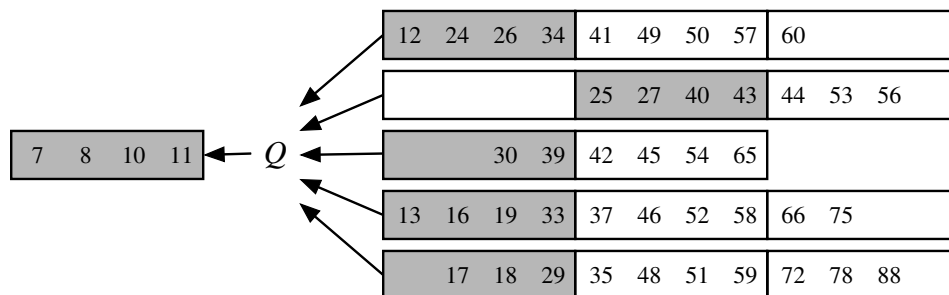


Figure 15.5: A d -way merge with $d = 5$ and $B = 4$. Blocks that currently reside in main memory are shaded.

We maintain the smallest unprocessed element from each input sequence in main memory, requesting the next block from a sequence when the preceding block has been exhausted. Similarly, we use one block of internal memory to buffer the merged sequence, flushing that block to external memory when full. In this way, the total number of transfers performed during a single d -way merge is $O(n/B)$, since we scan each block of list S_i once, and we write out each block of the merged list S' once. In terms of computation time, choosing the smallest of d values can trivially be performed using $O(d)$ operations. If we are willing to devote $O(d)$ internal memory, we can maintain a priority queue identifying the smallest element from each sequence, thereby performing each step of the merge in $O(\log d)$ time by removing the minimum element and replacing it with the next element from the same sequence. Hence, the internal time for the d -way merge is $O(n \log d)$.

Proposition 15.3: *Given an array-based sequence S of n elements stored compactly in external memory, we can sort S with $O((n/B) \log(n/B) / \log(M/B))$ block transfers and $O(n \log n)$ internal computations, where M is the size of the internal memory and B is the size of a block.*

Chapter 2

Divide-and-conquer algorithms

The *divide-and-conquer* strategy solves a problem by:

1. Breaking it into *subproblems* that are themselves smaller instances of the same type of problem
2. Recursively solving these subproblems
3. Appropriately combining their answers

The real work is done piecemeal, in three different places: in the partitioning of problems into subproblems; at the very tail end of the recursion, when the subproblems are so small that they are solved outright; and in the gluing together of partial answers. These are held together and coordinated by the algorithm's core recursive structure.

As an introductory example, we'll see how this technique yields a new algorithm for multiplying numbers, one that is much more efficient than the method we all learned in elementary school!

2.1 Multiplication

The mathematician Carl Friedrich Gauss (1777–1855) once noticed that although the product of two complex numbers

$$(a + bi)(c + di) = ac - bd + (bc + ad)i$$

seems to involve *four* real-number multiplications, it can in fact be done with just *three*: ac , bd , and $(a + b)(c + d)$, since

$$bc + ad = (a + b)(c + d) - ac - bd.$$

In our big- O way of thinking, reducing the number of multiplications from four to three seems wasted ingenuity. But this modest improvement becomes very significant *when applied recursively*.

Let's move away from complex numbers and see how this helps with regular multiplication. Suppose x and y are two n -bit integers, and assume for convenience that n is a power of 2 (the more general case is hardly any different). As a first step toward multiplying x and y , split each of them into their left and right halves, which are $n/2$ bits long:

$$x = \boxed{x_L} \boxed{x_R} = 2^{n/2}x_L + x_R$$

$$y = \boxed{y_L} \boxed{y_R} = 2^{n/2}y_L + y_R.$$

For instance, if $x = 10110110_2$ (the subscript 2 means “binary”) then $x_L = 1011_2$, $x_R = 0110_2$, and $x = 1011_2 \times 2^4 + 0110_2$. The product of x and y can then be rewritten as

$$xy = (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) = 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R.$$

We will compute xy via the expression on the right. The additions take linear time, as do the multiplications by powers of 2 (which are merely left-shifts). The significant operations are the four $n/2$ -bit multiplications, $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$; these we can handle by four recursive calls. Thus our method for multiplying n -bit numbers starts by making recursive calls to multiply these four pairs of $n/2$ -bit numbers (four subproblems of half the size), and then evaluates the preceding expression in $O(n)$ time. Writing $T(n)$ for the overall running time on n -bit inputs, we get the *recurrence relation*

$$T(n) = 4T(n/2) + O(n).$$

We will soon see general strategies for solving such equations. In the meantime, this particular one works out to $O(n^2)$, the same running time as the traditional grade-school multiplication technique. So we have a radically new algorithm, but we haven’t yet made any progress in efficiency. How can our method be sped up?

This is where Gauss’s trick comes to mind. Although the expression for xy seems to demand four $n/2$ -bit multiplications, as before just three will do: $x_L y_L$, $x_R y_R$, and $(x_L + x_R)(y_L + y_R)$, since $x_L y_R + x_R y_L = (x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R$. The resulting algorithm, shown in Figure 2.1, has an improved running time of¹

$$T(n) = 3T(n/2) + O(n).$$

The point is that now the constant factor improvement, from 4 to 3, occurs *at every level of the recursion*, and this compounding effect leads to a dramatically lower time bound of $O(n^{1.59})$.

This running time can be derived by looking at the algorithm’s pattern of recursive calls, which form a tree structure, as in Figure 2.2. Let’s try to understand the shape of this tree. At each successive level of recursion the subproblems get halved in size. At the $(\log_2 n)^{\text{th}}$ level, the subproblems get down to size 1, and so the recursion ends. Therefore, the height of the tree is $\log_2 n$. The branching factor is 3—each problem recursively produces three smaller ones—with the result that at depth k in the tree there are 3^k subproblems, each of size $n/2^k$.

For each subproblem, a linear amount of work is done in identifying further subproblems and combining their answers. Therefore the total time spent at depth k in the tree is

$$3^k \times O\left(\frac{n}{2^k}\right) = \left(\frac{3}{2}\right)^k \times O(n).$$

¹Actually, the recurrence should read

$$T(n) \leq 3T(n/2 + 1) + O(n)$$

since the numbers $(x_L + x_R)$ and $(y_L + y_R)$ could be $n/2 + 1$ bits long. The one we’re using is simpler to deal with and can be seen to imply exactly the same big- O running time.

Figure 2.1 A divide-and-conquer algorithm for integer multiplication.

```
function multiply( $x, y$ )  
Input:  Positive integers  $x$  and  $y$ , in binary  
Output: Their product  
  
 $n = \max(\text{size of } x, \text{size of } y)$   
if  $n = 1$ : return  $xy$   
  
 $x_L, x_R = \text{leftmost } \lceil n/2 \rceil, \text{rightmost } \lfloor n/2 \rfloor \text{ bits of } x$   
 $y_L, y_R = \text{leftmost } \lceil n/2 \rceil, \text{rightmost } \lfloor n/2 \rfloor \text{ bits of } y$   
  
 $P_1 = \text{multiply}(x_L, y_L)$   
 $P_2 = \text{multiply}(x_R, y_R)$   
 $P_3 = \text{multiply}(x_L + x_R, y_L + y_R)$   
return  $P_1 \times 2^n + (P_3 - P_1 - P_2) \times 2^{n/2} + P_2$ 
```

At the very top level, when $k = 0$, this works out to $O(n)$. At the bottom, when $k = \log_2 n$, it is $O(3^{\log_2 n})$, which can be rewritten as $O(n^{\log_2 3})$ (do you see why?). Between these two endpoints, the work done increases *geometrically* from $O(n)$ to $O(n^{\log_2 3})$, by a factor of $3/2$ per level. The sum of any increasing geometric series is, within a constant factor, simply the last term of the series: such is the rapidity of the increase (Exercise 0.2). Therefore the overall running time is $O(n^{\log_2 3})$, which is about $O(n^{1.59})$.

In the absence of Gauss's trick, the recursion tree would have the same height, but the branching factor would be 4. There would be $4^{\log_2 n} = n^2$ leaves, and therefore the running time would be at least this much. In divide-and-conquer algorithms, the number of subproblems translates into the branching factor of the recursion tree; small changes in this coefficient can have a big impact on running time.

A practical note: it generally does not make sense to recurse all the way down to 1 bit. For most processors, 16- or 32-bit multiplication is a single operation, so by the time the numbers get into this range they should be handed over to the built-in procedure.

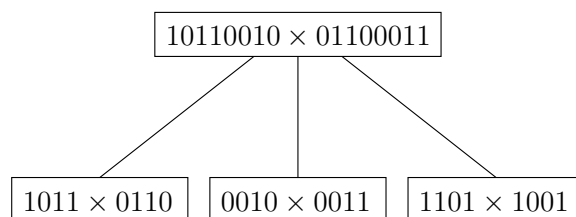
Finally, the eternal question: *Can we do better?* It turns out that even faster algorithms for multiplying numbers exist, based on another important divide-and-conquer algorithm: the fast Fourier transform, to be explained in Section 2.6.

2.2 Recurrence relations

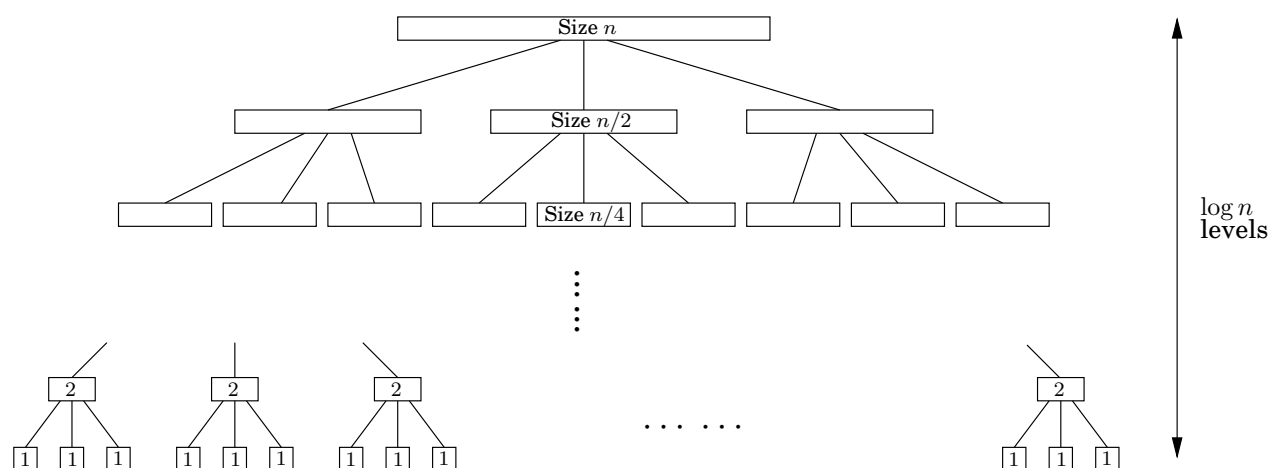
Divide-and-conquer algorithms often follow a generic pattern: they tackle a problem of size n by recursively solving, say, a subproblems of size n/b and then combining these answers in $O(n^d)$ time, for some $a, b, d > 0$ (in the multiplication algorithm, $a = 3$, $b = 2$, and $d = 1$). Their running time can therefore be captured by the equation $T(n) = aT(\lceil n/b \rceil) + O(n^d)$. We next derive a closed-form solution to this general recurrence so that we no longer have to solve it explicitly in each new instance.

Figure 2.2 Divide-and-conquer integer multiplication. (a) Each problem is divided into three subproblems. (b) The levels of recursion.

(a)



(b)



Master theorem² If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for some constants $a > 0$, $b > 1$, and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a. \end{cases}$$

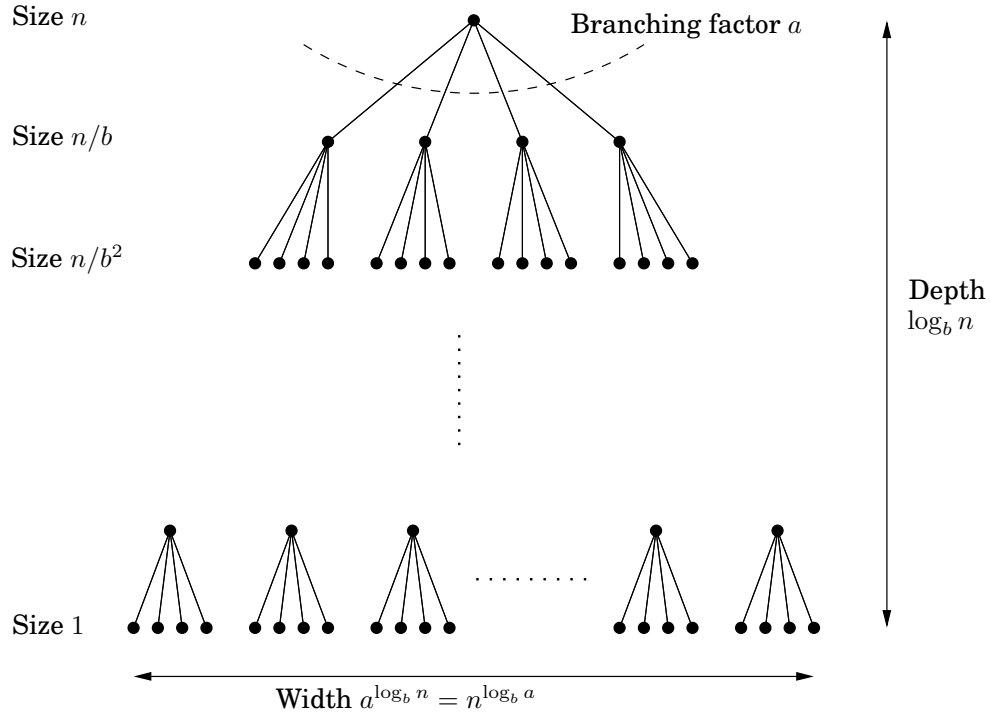
This single theorem tells us the running times of most of the divide-and-conquer procedures we are likely to use.

Proof. To prove the claim, let's start by assuming for the sake of convenience that n is a power of b . This will not influence the final bound in any important way—after all, n is at most a multiplicative factor of b away from some power of b (Exercise 2.2)—and it will allow us to ignore the rounding effect in $\lceil n/b \rceil$.

Next, notice that the size of the subproblems decreases by a factor of b with each level of recursion, and therefore reaches the base case after $\log_b n$ levels. This is the height of the recursion tree. Its branching factor is a , so the k th level of the tree is made up of a^k

²There are even more general results of this type, but we will not be needing them.

Figure 2.3 Each problem of size n is divided into a subproblems of size n/b .



subproblems, each of size n/b^k (Figure 2.3). The total work done at this level is

$$a^k \times O\left(\frac{n}{b^k}\right)^d = O(n^d) \times \left(\frac{a}{b^d}\right)^k.$$

As k goes from 0 (the root) to $\log_b n$ (the leaves), these numbers form a geometric series with ratio a/b^d . Finding the sum of such a series in big- O notation is easy (Exercise 0.2), and comes down to three cases.

1. *The ratio is less than 1.*

Then the series is decreasing, and its sum is just given by its first term, $O(n^d)$.

2. *The ratio is greater than 1.*

The series is increasing and its sum is given by its last term, $O(n^{\log_b a})$:

$$n^d \left(\frac{a}{b^d}\right)^{\log_b n} = n^d \left(\frac{a^{\log_b n}}{(b^{\log_b n})^d}\right) = a^{\log_b n} = a^{(\log_a n)(\log_b a)} = n^{\log_b a}.$$

3. *The ratio is exactly 1.*

In this case all $O(\log n)$ terms of the series are equal to $O(n^d)$.

These cases translate directly into the three contingencies in the theorem statement. ■

Binary search

The ultimate divide-and-conquer algorithm is, of course, *binary search*: to find a key k in a large file containing keys $z[0, 1, \dots, n-1]$ in sorted order, we first compare k with $z[\lceil n/2 \rceil]$, and depending on the result we recurse either on the first half of the file, $z[0, \dots, n/2 - 1]$, or on the second half, $z[n/2, \dots, n-1]$. The recurrence now is $T(n) = T(\lceil n/2 \rceil) + O(1)$, which is the case $a = 1, b = 2, d = 0$. Plugging into our master theorem we get the familiar solution: a running time of just $O(\log n)$.

2.3 Mergesort

The problem of sorting a list of numbers lends itself immediately to a divide-and-conquer strategy: split the list into two halves, recursively sort each half, and then *merge* the two sorted sublists.

```
function mergesort( $a[1 \dots n]$ )
Input:  An array of numbers  $a[1 \dots n]$ 
Output: A sorted version of this array

if  $n > 1$ :
    return merge(mergesort( $a[1 \dots \lfloor n/2 \rfloor]$ ), mergesort( $a[\lfloor n/2 \rfloor + 1 \dots n]$ ))
else:
    return  $a$ 
```

The correctness of this algorithm is self-evident, as long as a correct merge subroutine is specified. If we are given two sorted arrays $x[1 \dots k]$ and $y[1 \dots l]$, how do we efficiently merge them into a single sorted array $z[1 \dots k + l]$? Well, the very first element of z is either $x[1]$ or $y[1]$, whichever is smaller. The rest of $z[\cdot]$ can then be constructed recursively.

```
function merge( $x[1 \dots k], y[1 \dots l]$ )
if  $k = 0$ : return  $y[1 \dots l]$ 
if  $l = 0$ : return  $x[1 \dots k]$ 
if  $x[1] \leq y[1]$ :
    return  $x[1] \circ \text{merge}(x[2 \dots k], y[1 \dots l])$ 
else:
    return  $y[1] \circ \text{merge}(x[1 \dots k], y[2 \dots l])$ 
```

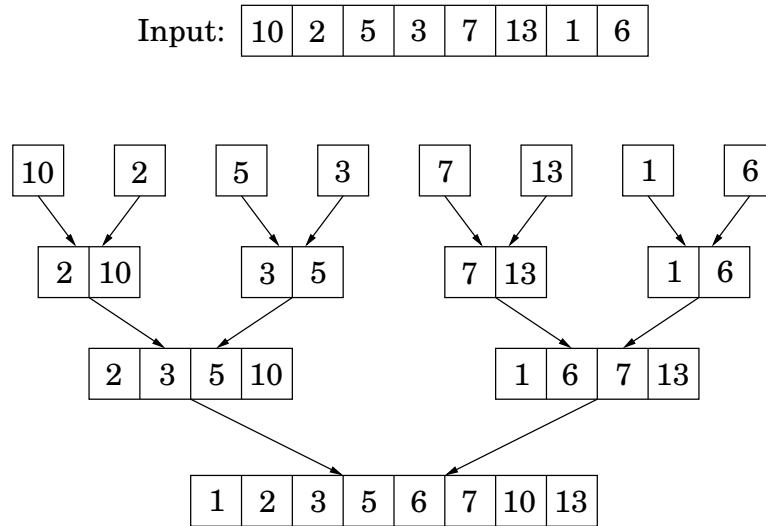
Here $\circ$ denotes concatenation. This merge procedure does a constant amount of work per recursive call (provided the required array space is allocated in advance), for a total running time of $O(k + l)$. Thus merge's are linear, and the overall time taken by mergesort is

$$T(n) = 2T(n/2) + O(n),$$

or $O(n \log n)$.

Looking back at the mergesort algorithm, we see that all the real work is done in merging, which doesn't start until the recursion gets down to singleton arrays. The singletons are

Figure 2.4 The sequence of merge operations in mergesort.



merged in pairs, to yield arrays with two elements. Then pairs of these 2-tuples are merged, producing 4-tuples, and so on. Figure 2.4 shows an example.

This viewpoint also suggests how mergesort might be made iterative. At any given moment, there is a set of “active” arrays—initially, the singletons—which are merged in pairs to give the next batch of active arrays. These arrays can be organized in a queue, and processed by repeatedly removing two arrays from the front of the queue, merging them, and putting the result at the end of the queue.

In the following pseudocode, the primitive operation `inject` adds an element to the end of the queue while `eject` removes and returns the element at the front of the queue.

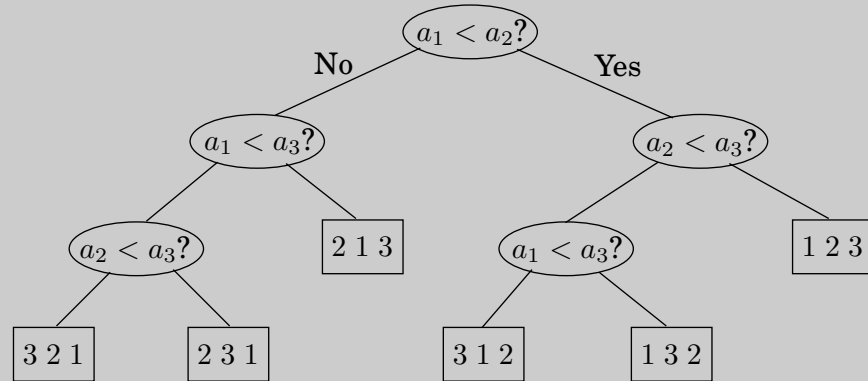
```

function iterative-mergesort( $a[1 \dots n]$ )
Input:  elements  $a_1, a_2, \dots, a_n$  to be sorted

 $Q = [ ]$  (empty queue)
for  $i = 1$  to  $n$ :
    inject( $Q, [a_i]$ )
while  $|Q| > 1$ :
    inject( $Q, \text{merge}(\text{eject}(Q), \text{eject}(Q))$ )
return eject( $Q$ )
  
```


An $n \log n$ lower bound for sorting

Sorting algorithms can be depicted as trees. The one in the following figure sorts an array of three elements, a_1, a_2, a_3 . It starts by comparing a_1 to a_2 and, if the first is larger, compares it with a_3 ; otherwise it compares a_2 and a_3 . And so on. Eventually we end up at a leaf, and this leaf is labeled with the true order of the three elements as a permutation of 1, 2, 3. For example, if $a_2 < a_1 < a_3$, we get the leaf labeled “2 1 3.”



The *depth* of the tree—the number of comparisons on the longest path from root to leaf, in this case 3—is exactly the worst-case time complexity of the algorithm.

This way of looking at sorting algorithms is useful because it allows one to argue that *mergesort is optimal*, in the sense that $\Omega(n \log n)$ comparisons are necessary for sorting n elements.

Here is the argument: Consider any such tree that sorts an array of n elements. Each of its leaves is labeled by a permutation of $\{1, 2, \dots, n\}$. In fact, *every* permutation must appear as the label of a leaf. The reason is simple: if a particular permutation is missing, what happens if we feed the algorithm an input ordered according to this same permutation? And since there are $n!$ permutations of n elements, it follows that the tree has at least $n!$ leaves.

We are almost done: This is a binary tree, and we argued that it has at least $n!$ leaves. Recall now that a binary tree of depth d has at most 2^d leaves (proof: an easy induction on d). So, the depth of our tree—and the complexity of our algorithm—must be at least $\log(n!)$.

And it is well known that $\log(n!) \geq c \cdot n \log n$ for some $c > 0$. There are many ways to see this. The easiest is to notice that $n! \geq (n/2)^{(n/2)}$ because $n! = 1 \cdot 2 \cdot \dots \cdot n$ contains at least $n/2$ factors larger than $n/2$; and to then take logs of both sides. Another is to recall Stirling’s formula

$$n! \approx \sqrt{\pi \left(2n + \frac{1}{3}\right)} \cdot n^n \cdot e^{-n}.$$

Either way, we have established that any comparison tree that sorts n elements must make, in the worst case, $\Omega(n \log n)$ comparisons, and hence mergesort is optimal!

Well, there is some fine print: this neat argument applies only to *algorithms that use comparisons*. Is it conceivable that there are alternative sorting strategies, perhaps using sophisticated numerical manipulations, that work in linear time? The answer is *yes*, under certain exceptional circumstances: the canonical such example is when the elements to be sorted are integers that lie in a small range (Exercise 2.20).

2.4 Medians

The *median* of a list of numbers is its 50th percentile: half the numbers are bigger than it, and half are smaller. For instance, the median of $[45, 1, 10, 30, 25]$ is 25, since this is the middle element when the numbers are arranged in order. If the list has even length, there are two choices for what the middle element could be, in which case we pick the smaller of the two, say.

The purpose of the median is to summarize a set of numbers by a single, typical value. The *mean*, or average, is also very commonly used for this, but the median is in a sense more typical of the data: it is always one of the data values, unlike the mean, and it is less sensitive to outliers. For instance, the median of a list of a hundred 1's is (rightly) 1, as is the mean. However, if just one of these numbers gets accidentally corrupted to 10,000, the mean shoots up above 100, while the median is unaffected.

Computing the median of n numbers is easy: just sort them. The drawback is that this takes $O(n \log n)$ time, whereas we would ideally like something linear. We have reason to be hopeful, because sorting is doing far more work than we really need—we just want the middle element and don't care about the relative ordering of the rest of them.

When looking for a recursive solution, it is paradoxically often easier to work with a *more general* version of the problem—for the simple reason that this gives a more powerful step to recurse upon. In our case, the generalization we will consider is *selection*.

SELECTION

Input: A list of numbers S ; an integer k

Output: The k th smallest element of S

For instance, if $k = 1$, the minimum of S is sought, whereas if $k = \lfloor |S|/2 \rfloor$, it is the median.

A randomized divide-and-conquer algorithm for selection

Here's a divide-and-conquer approach to selection. For any number v , imagine splitting list S into three categories: elements smaller than v , those equal to v (there might be duplicates), and those greater than v . Call these S_L , S_v , and S_R respectively. For instance, if the array

$S :$

2	36	5	21	8	13	11	20	5	4	1
---	----	---	----	---	----	----	----	---	---	---

is split on $v = 5$, the three subarrays generated are

$S_L :$

2	4	1
---	---	---

 $S_v :$

5	5
---	---

 $S_R :$

36	21	8	13	11	20
----	----	---	----	----	----

The search can instantly be narrowed down to one of these sublists. If we want, say, the *eighth*-smallest element of S , we know it must be the *third*-smallest element of S_R since $|S_L| + |S_v| = 5$. That is, $\text{selection}(S, 8) = \text{selection}(S_R, 3)$. More generally, by checking k against the sizes of the subarrays, we can quickly determine which of them holds the desired element:

$$\text{selection}(S, k) = \begin{cases} \text{selection}(S_L, k) & \text{if } k \leq |S_L| \\ v & \text{if } |S_L| < k \leq |S_L| + |S_v| \\ \text{selection}(S_R, k - |S_L| - |S_v|) & \text{if } k > |S_L| + |S_v|. \end{cases}$$

The three sublists S_L , S_v , and S_R can be computed from S in linear time; in fact, this computation can even be done *in place*, that is, without allocating new memory (Exercise 2.15). We then recurse on the appropriate sublist. The effect of the split is thus to shrink the number of elements from $|S|$ to at most $\max\{|S_L|, |S_R|\}$.

Our divide-and-conquer algorithm for selection is now fully specified, except for the crucial detail of how to choose v . It should be picked quickly, and it should shrink the array substantially, the ideal situation being $|S_L|, |S_R| \approx \frac{1}{2}|S|$. If we could always guarantee this situation, we would get a running time of

$$T(n) = T(n/2) + O(n),$$

which is linear as desired. But this requires picking v to be the median, which is our ultimate goal! Instead, we follow a much simpler alternative: *we pick v randomly from S .*

Efficiency analysis

Naturally, the running time of our algorithm depends on the random choices of v . It is possible that due to persistent bad luck we keep picking v to be the largest element of the array (or the smallest element), and thereby shrink the array by only one element each time. In the earlier example, we might first pick $v = 36$, then $v = 21$, and so on. This *worst-case* scenario would force our selection algorithm to perform

$$n + (n - 1) + (n - 2) + \cdots + \frac{n}{2} = \Theta(n^2)$$

operations (when computing the median), but it is extremely unlikely to occur. Equally unlikely is the *best* possible case we discussed before, in which each randomly chosen v just happens to split the array perfectly in half, resulting in a running time of $O(n)$. Where, in this spectrum from $O(n)$ to $\Theta(n^2)$, does the *average* running time lie? Fortunately, it lies very close to the best-case time.

To distinguish between lucky and unlucky choices of v , we will call v *good* if it lies within the 25th to 75th percentile of the array that it is chosen from. We like these choices of v because they ensure that the sublists S_L and S_R have size at most three-fourths that of S (do you see why?), so that the array shrinks substantially. Fortunately, good v 's are abundant: half the elements of any list must fall between the 25th to 75th percentile!

Given that a randomly chosen v has a 50% chance of being good, how many v 's do we need to pick on average before getting a good one? Here's a more familiar reformulation (see also Exercise 1.34):

Lemma *On average a fair coin needs to be tossed two times before a "heads" is seen.*

Proof. Let E be the expected number of tosses before a heads is seen. We certainly need at least one toss, and if it's heads, we're done. If it's tails (which occurs with probability $1/2$), we need to repeat. Hence $E = 1 + \frac{1}{2}E$, which works out to $E = 2$. ■

Therefore, after two split operations on average, the array will shrink to at most three-fourths of its size. Letting $T(n)$ be the *expected* running time on an array of size n , we get

$$T(n) \leq T(3n/4) + O(n).$$

This follows by taking expected values of both sides of the following statement:

$$\begin{aligned} &\text{Time taken on an array of size } n \\ &\leq (\text{time taken on an array of size } 3n/4) + (\text{time to reduce array size to } \leq 3n/4), \end{aligned}$$

and, for the right-hand side, using the familiar property that *the expectation of the sum is the sum of the expectations*.

From this recurrence we conclude that $T(n) = O(n)$: on *any* input, our algorithm returns the correct answer after a linear number of steps, on the average.

The Unix sort command

Comparing the algorithms for sorting and median-finding we notice that, beyond the common divide-and-conquer philosophy and structure, they are exact opposites. Mergesort splits the array in two in the most convenient way (first half, second half), without any regard to the magnitudes of the elements in each half; but then it works hard to put the sorted subarrays together. In contrast, the median algorithm is careful about its splitting (smaller numbers first, then the larger ones), but its work ends with the recursive call.

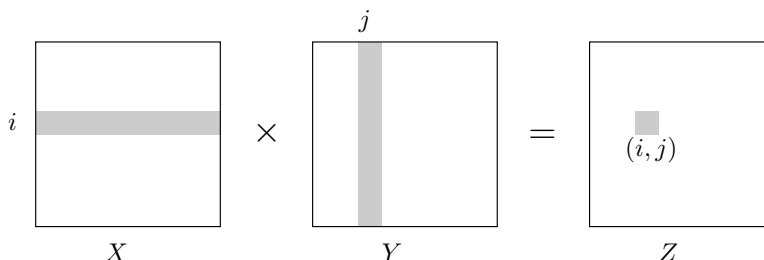
Quicksort is a sorting algorithm that splits the array in exactly the same way as the median algorithm; and once the subarrays are sorted, by two recursive calls, there is nothing more to do. Its worst-case performance is $\Theta(n^2)$, like that of median-finding. But it can be proved (Exercise 2.24) that its average case is $O(n \log n)$; furthermore, empirically it outperforms other sorting algorithms. This has made quicksort a favorite in many applications—for instance, it is the basis of the code by which really enormous files are sorted.

2.5 Matrix multiplication

The product of two $n \times n$ matrices X and Y is a third $n \times n$ matrix $Z = XY$, with (i, j) th entry

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}.$$

To make it more visual, Z_{ij} is the dot product of the i th row of X with the j th column of Y :



In general, XY is not the same as YX ; matrix multiplication is not commutative.

The preceding formula implies an $O(n^3)$ algorithm for matrix multiplication: there are n^2 entries to be computed, and each takes $O(n)$ time. For quite a while, this was widely believed

to be the best running time possible, and it was even proved that in certain models of computation no algorithm could do better. It was therefore a source of great excitement when in 1969, the German mathematician Volker Strassen announced a significantly more efficient algorithm, based upon divide-and-conquer.

Matrix multiplication is particularly easy to break into subproblems, because it can be performed *blockwise*. To see what this means, carve X into four $n/2 \times n/2$ blocks, and also Y :

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}.$$

Then their product can be expressed in terms of these blocks and is exactly as if the blocks were single elements (Exercise 2.11).

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

We now have a divide-and-conquer strategy: to compute the size- n product XY , recursively compute eight size- $n/2$ products $AE, BG, AF, BH, CE, DG, CF, DH$, and then do a few $O(n^2)$ -time additions. The total running time is described by the recurrence relation

$$T(n) = 8T(n/2) + O(n^2).$$

This comes out to an unimpressive $O(n^3)$, the same as for the default algorithm. But the efficiency *can* be further improved, and as with integer multiplication, the key is some clever algebra. It turns out XY can be computed from just *seven* $n/2 \times n/2$ subproblems, via a decomposition so tricky and intricate that one wonders how Strassen was ever able to discover it!

$$XY = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

where

$$\begin{array}{ll} P_1 &= A(F - H) & P_5 &= (A + D)(E + H) \\ P_2 &= (A + B)H & P_6 &= (B - D)(G + H) \\ P_3 &= (C + D)E & P_7 &= (A - C)(E + F) \\ P_4 &= D(G - E) \end{array}$$

The new running time is

$$T(n) = 7T(n/2) + O(n^2),$$

which by the master theorem works out to $O(n^{\log_2 7}) \approx O(n^{2.81})$.

2.6 The fast Fourier transform

We have so far seen how divide-and-conquer gives fast algorithms for multiplying integers and matrices; our next target is *polynomials*. The product of two degree- d polynomials is a polynomial of degree $2d$, for example:

$$(1 + 2x + 3x^2) \cdot (2 + x + 4x^2) = 2 + 5x + 12x^2 + 11x^3 + 12x^4.$$

More generally, if $A(x) = a_0 + a_1x + \cdots + a_dx^d$ and $B(x) = b_0 + b_1x + \cdots + b_dx^d$, their product $C(x) = A(x) \cdot B(x) = c_0 + c_1x + \cdots + c_{2d}x^{2d}$ has coefficients

$$c_k = a_0b_k + a_1b_{k-1} + \cdots + a_kb_0 = \sum_{i=0}^k a_ib_{k-i}$$

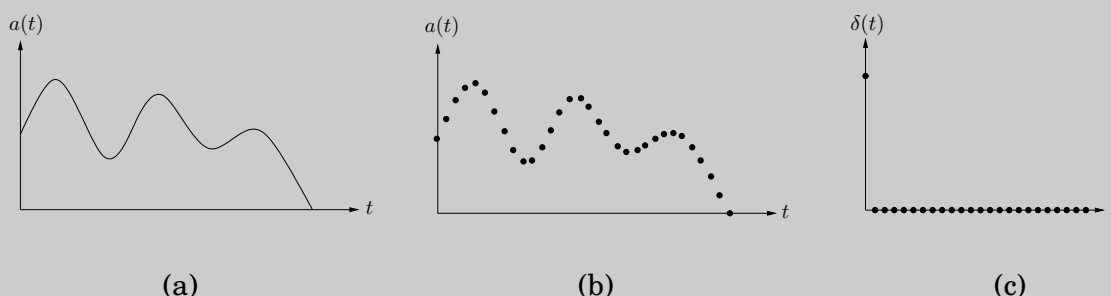
(for $i > d$, take a_i and b_i to be zero). Computing c_k from this formula takes $O(k)$ steps, and finding all $2d + 1$ coefficients would therefore seem to require $\Theta(d^2)$ time. *Can we possibly multiply polynomials faster than this?*

The solution we will develop, the fast Fourier transform, has revolutionized—indeed, defined—the field of signal processing (see the following box). Because of its huge importance, and its wealth of insights from different fields of study, we will approach it a little more leisurely than usual. The reader who wants just the core algorithm can skip directly to Section 2.6.4.

Why multiply polynomials?

For one thing, it turns out that the fastest algorithms we have for multiplying integers rely heavily on polynomial multiplication; after all, polynomials and binary integers are quite similar—just replace the variable x by the base 2, and watch out for carries. But perhaps more importantly, multiplying polynomials is crucial for *signal processing*.

A *signal* is any quantity that is a function of time (as in Figure (a)) or of position. It might, for instance, capture a human voice by measuring fluctuations in air pressure close to the speaker’s mouth, or alternatively, the pattern of stars in the night sky, by measuring brightness as a function of angle.



In order to extract information from a signal, we need to first *digitize* it by sampling (Figure (b))—and, then, to put it through a *system* that will transform it in some way. The output is called the *response* of the system:

$$\text{signal} \longrightarrow \boxed{\text{SYSTEM}} \longrightarrow \text{response}$$

An important class of systems are those that are *linear*—the response to the sum of two signals is just the sum of their individual responses—and *time invariant*—shifting the input signal by time t produces the same output, also shifted by t . Any system with these properties is completely characterized by its response to the simplest possible input signal: the *unit impulse* $\delta(t)$, consisting solely of a “jerk” at $t = 0$ (Figure (c)). To see this, first consider the close relative $\delta(t - i)$, a shifted impulse in which the jerk occurs at time i . Any signal $a(t)$ can be expressed as a linear combination of these, letting $\delta(t - i)$ pick out its behavior at time i ,

$$a(t) = \sum_{i=0}^{T-1} a(i)\delta(t - i)$$

(if the signal consists of T samples). By linearity, the system response to input $a(t)$ is determined by the responses to the various $\delta(t - i)$. And by time invariance, these are in turn just shifted copies of the *impulse response* $b(t)$, the response to $\delta(t)$.

In other words, the output of the system at time k is

$$c(k) = \sum_{i=0}^k a(i)b(k - i),$$

exactly the formula for polynomial multiplication!

2.6.1 An alternative representation of polynomials

To arrive at a fast algorithm for polynomial multiplication we take inspiration from an important property of polynomials.

Fact A degree- d polynomial is uniquely characterized by its values at any $d + 1$ distinct points.

A familiar instance of this is that “any two points determine a line.” We will later see why the more general statement is true (page 72), but for the time being it gives us an *alternative representation* of polynomials. Fix any distinct points $x_0, \dots, x_d$. We can specify a degree- d polynomial $A(x) = a_0 + a_1x + \dots + a_dx^d$ by either one of the following:

1. Its coefficients $a_0, a_1, \dots, a_d$
2. The values $A(x_0), A(x_1), \dots, A(x_d)$

Of these two representations, the second is the more attractive for polynomial multiplication. Since the product $C(x)$ has degree $2d$, it is completely determined by its value at any $2d + 1$ points. And its value at any given point z is easy enough to figure out, just $A(z)$ times $B(z)$. Thus *polynomial multiplication takes linear time in the value representation*.

The problem is that we expect the input polynomials, and also their product, to be specified by coefficients. So we need to first translate from coefficients to values—which is just a matter of *evaluating* the polynomial at the chosen points—then multiply in the value representation, and finally translate back to coefficients, a process called *interpolation*.

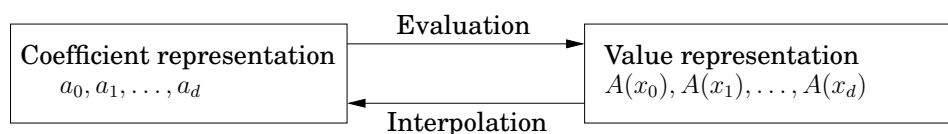


Figure 2.5 presents the resulting algorithm.

Figure 2.5 Polynomial multiplication

Input: Coefficients of two polynomials, $A(x)$ and $B(x)$, of degree d

Output: Their product $C = A \cdot B$

Selection

Pick some points $x_0, x_1, \dots, x_{n-1}$, where $n \geq 2d + 1$

Evaluation

Compute $A(x_0), A(x_1), \dots, A(x_{n-1})$ and $B(x_0), B(x_1), \dots, B(x_{n-1})$

Multiplication

Compute $C(x_k) = A(x_k)B(x_k)$ for all $k = 0, \dots, n - 1$

Interpolation

Recover $C(x) = c_0 + c_1x + \dots + c_{2d}x^{2d}$

The equivalence of the two polynomial representations makes it clear that this high-level approach is correct, but how efficient is it? Certainly the selection step and the n multiplications are no trouble at all, just linear time.³ But (leaving aside interpolation, about which we know even less) how about evaluation? Evaluating a polynomial of degree $d \leq n$ at a single point takes $O(n)$ steps (Exercise 2.29), and so the baseline for n points is $\Theta(n^2)$. We'll now see that the fast Fourier transform (FFT) does it in just $O(n \log n)$ time, for a particularly clever choice of $x_0, \dots, x_{n-1}$ in which the computations required by the individual points overlap with one another and can be shared.

2.6.2 Evaluation by divide-and-conquer

Here's an idea for how to pick the n points at which to evaluate a polynomial $A(x)$ of degree $\leq n-1$. If we choose them to be positive-negative pairs, that is,

$$\pm x_0, \pm x_1, \dots, \pm x_{n/2-1},$$

then the computations required for each $A(x_i)$ and $A(-x_i)$ overlap a lot, because the even powers of x_i coincide with those of $-x_i$.

To investigate this, we need to split $A(x)$ into its odd and even powers, for instance

$$3 + 4x + 6x^2 + 2x^3 + x^4 + 10x^5 = (3 + 6x^2 + x^4) + x(4 + 2x^2 + 10x^4).$$

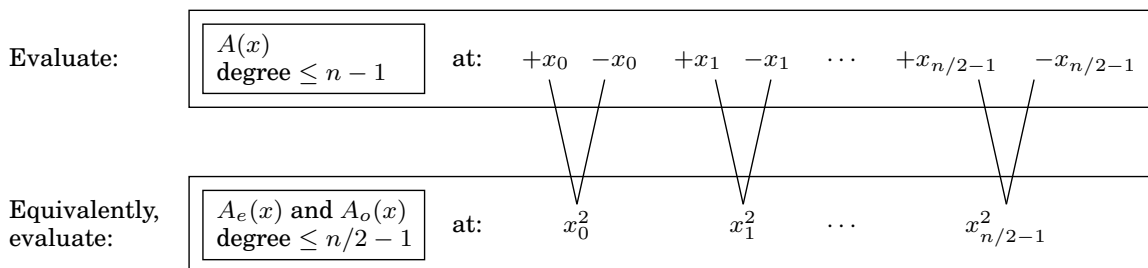
Notice that the terms in parentheses are polynomials in x^2 . More generally,

$$A(x) = A_e(x^2) + xA_o(x^2),$$

where $A_e(\cdot)$, with the even-numbered coefficients, and $A_o(\cdot)$, with the odd-numbered coefficients, are polynomials of degree $\leq n/2 - 1$ (assume for convenience that n is even). Given *paired* points $\pm x_i$, the calculations needed for $A(x_i)$ can be recycled toward computing $A(-x_i)$:

$$\begin{aligned} A(x_i) &= A_e(x_i^2) + x_i A_o(x_i^2) \\ A(-x_i) &= A_e(x_i^2) - x_i A_o(x_i^2). \end{aligned}$$

In other words, evaluating $A(x)$ at n paired points $\pm x_0, \dots, \pm x_{n/2-1}$ reduces to evaluating $A_e(x)$ and $A_o(x)$ (which each have half the degree of $A(x)$) at just $n/2$ points, $x_0^2, \dots, x_{n/2-1}^2$.



³In a typical setting for polynomial multiplication, the coefficients of the polynomials are real numbers and, moreover, are small enough that basic arithmetic operations (adding and multiplying) take unit time. We will assume this to be the case without any great loss of generality; in particular, the time bounds we obtain are easily adjustable to situations with larger numbers.

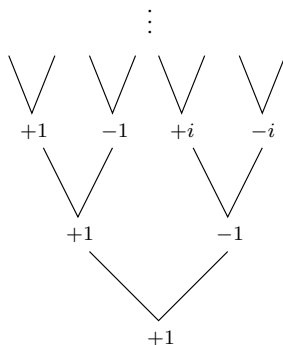
The original problem of size n is in this way recast as two subproblems of size $n/2$, followed by some linear-time arithmetic. If we could recurse, we would get a divide-and-conquer procedure with running time

$$T(n) = 2T(n/2) + O(n),$$

which is $O(n \log n)$, exactly what we want.

But we have a problem: The plus-minus trick only works at the top level of the recursion. To recurse at the next level, we need the $n/2$ evaluation points $x_0^2, x_1^2, \dots, x_{n/2-1}^2$ to be *themselves* plus-minus pairs. But how can a square be negative? The task seems impossible! *Unless, of course, we use complex numbers.*

Fine, but which complex numbers? To figure this out, let us “reverse engineer” the process. At the very bottom of the recursion, we have a single point. This point might as well be 1, in which case the level above it must consist of its square roots, $\pm\sqrt{1} = \pm 1$.



The next level up then has $\pm\sqrt{+1} = \pm 1$ as well as the *complex* numbers $\pm\sqrt{-1} = \pm i$, where i is the imaginary unit. By continuing in this manner, we eventually reach the initial set of n points. Perhaps you have already guessed what they are: the *complex n th roots of unity*, that is, the n complex solutions to the equation $z^n = 1$.

Figure 2.6 is a pictorial review of some basic facts about complex numbers. The third panel of this figure introduces the n th roots of unity: the complex numbers $1, \omega, \omega^2, \dots, \omega^{n-1}$, where $\omega = e^{2\pi i/n}$. If n is even,

1. The n th roots are plus-minus paired, $\omega^{n/2+j} = -\omega^j$.
2. Squaring them produces the $(n/2)$ nd roots of unity.

Therefore, if we start with these numbers for some n that is a power of 2, then at successive levels of recursion we will have the $(n/2^k)$ th roots of unity, for $k = 0, 1, 2, 3, \dots$. All these sets of numbers are plus-minus paired, and so our divide-and-conquer, as shown in the last panel, works perfectly. The resulting algorithm is the fast Fourier transform (Figure 2.7).

Figure 2.6 The complex roots of unity are ideal for our divide-and-conquer scheme.

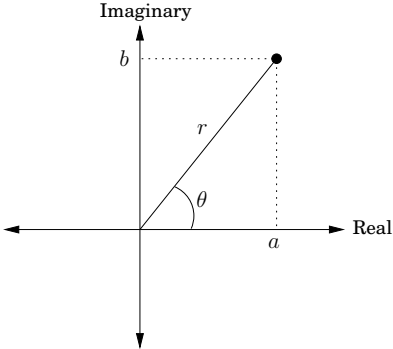
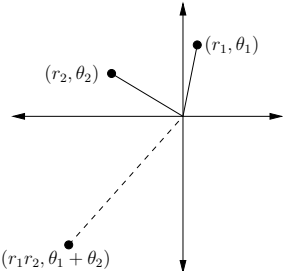
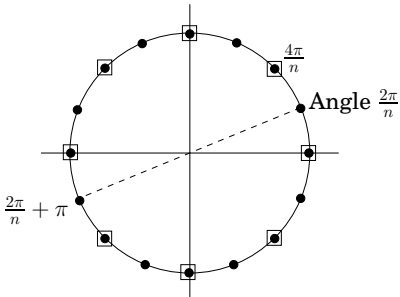
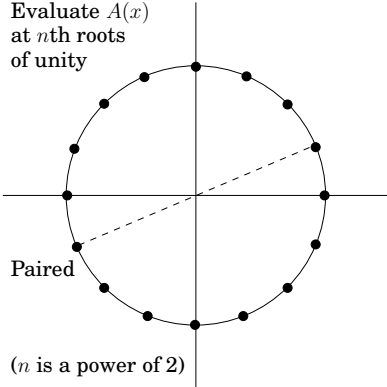
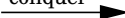
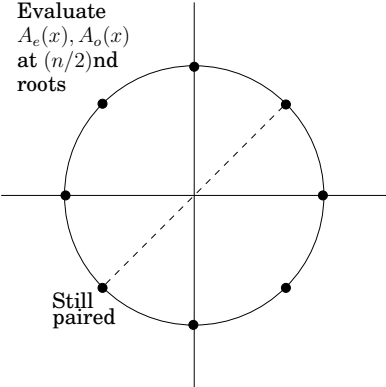
	The complex plane $z = a + bi$ is plotted at position (a, b). Polar coordinates: rewrite as $z = r(\cos \theta + i \sin \theta) = re^{i\theta}$, denoted (r, θ). • <i>length</i> $r = \sqrt{a^2 + b^2}$. • <i>angle</i> $\theta \in [0, 2\pi)$: $\cos \theta = a/r, \sin \theta = b/r$. • θ can always be reduced modulo 2π. Examples: <table><tr><th>Number</th><td>-1</td><td>i</td><td>5 + 5i</td></tr><tr><th>Polar coords</th><td>(1, π)</td><td>(1, $\pi/2$)</td><td>($5\sqrt{2}$, $\pi/4$)</td></tr></table>	Number	-1	i	5 + 5i	Polar coords	(1, π)	(1, $\pi/2$)	($5\sqrt{2}$, $\pi/4$)
Number	-1	i	5 + 5i						
Polar coords	(1, π)	(1, $\pi/2$)	($5\sqrt{2}$, $\pi/4$)						
	Multiplying is easy in polar coordinates Multiply the lengths and add the angles: $(r_1, \theta_1) \times (r_2, \theta_2) = (r_1 r_2, \theta_1 + \theta_2)$. For any $z = (r, \theta)$, • $-z = (r, \theta + \pi)$ since $-1 = (1, \pi)$. • If z is on the <i>unit circle</i> (i.e., $r = 1$), then $z^n = (1, n\theta)$.								
	The nth complex roots of unity Solutions to the equation $z^n = 1$. By the multiplication rule: solutions are $z = (1, \theta)$, for θ a multiple of $2\pi/n$ (shown here for $n = 16$). For even n: • These numbers are <i>plus-minus paired</i>: $-(1, \theta) = (1, \theta + \pi)$. • Their squares are the $(n/2)$nd roots of unity, shown here with boxes around them.								
Divide-and-conquer step Evaluate $A(x)$ at nth roots of unity  Paired (n is a power of 2) Divide and conquer  Evaluate $A_e(x), A_o(x)$ at $(n/2)$nd roots  Still paired									

Figure 2.7 The fast Fourier transform (polynomial formulation)

function FFT(A, ω)

Input: Coefficient representation of a polynomial $A(x)$
of degree $\leq n-1$, where n is a power of 2
 ω , an n th root of unity

Output: Value representation $A(\omega^0), \dots, A(\omega^{n-1})$

if $\omega = 1$: return $A(1)$

express $A(x)$ in the form $A_e(x^2) + xA_o(x^2)$

call FFT(A_e, ω^2) to evaluate A_e at even powers of ω

call FFT(A_o, ω^2) to evaluate A_o at even powers of ω

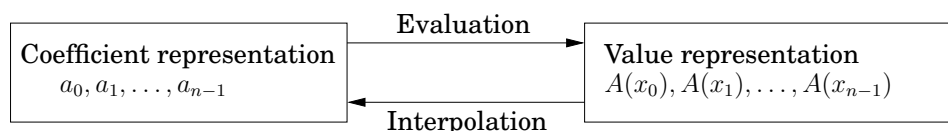
for $j = 0$ to $n-1$:

 compute $A(\omega^j) = A_e(\omega^{2j}) + \omega^j A_o(\omega^{2j})$

return $A(\omega^0), \dots, A(\omega^{n-1})$

2.6.3 Interpolation

Let's take stock of where we are. We first developed a high-level scheme for multiplying polynomials (Figure 2.5), based on the observation that polynomials can be represented in two ways, in terms of their *coefficients* or in terms of their *values* at a selected set of points.



The value representation makes it trivial to multiply polynomials, but we cannot ignore the coefficient representation since it is the form in which the input and output of our overall algorithm are specified.

So we designed the FFT, a way to move from coefficients to values in time just $O(n \log n)$, when the points $\{x_i\}$ are complex n th roots of unity $(1, \omega, \omega^2, \dots, \omega^{n-1})$.

$$\langle \text{values} \rangle = \text{FFT}(\langle \text{coefficients} \rangle, \omega).$$

The last remaining piece of the puzzle is the inverse operation, interpolation. It will turn out, amazingly, that

$$\langle \text{coefficients} \rangle = \frac{1}{n} \text{FFT}(\langle \text{values} \rangle, \omega^{-1}).$$

Interpolation is thus solved in the most simple and elegant way we could possibly have hoped for—using the same FFT algorithm, but called with ω^{-1} in place of ω ! This might seem like a miraculous coincidence, but it will make a lot more sense when we recast our polynomial operations in the language of linear algebra. Meanwhile, our $O(n \log n)$ polynomial multiplication algorithm (Figure 2.5) is now fully specified.

A matrix reformulation

To get a clearer view of interpolation, let's explicitly set down the relationship between our two representations for a polynomial $A(x)$ of degree $\leq n - 1$. They are both vectors of n numbers, and one is a linear transformation of the other:

$$\begin{bmatrix} A(x_0) \\ A(x_1) \\ \vdots \\ A(x_{n-1}) \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix}.$$

Call the matrix in the middle M . Its specialized format—a *Vandermonde* matrix—gives it many remarkable properties, of which the following is particularly relevant to us.

If $x_0, \dots, x_{n-1}$ are distinct numbers, then M is invertible.

The existence of M^{-1} allows us to invert the preceding matrix equation so as to express coefficients in terms of values. In brief,

Evaluation is multiplication by M , while interpolation is multiplication by M^{-1} .

This reformulation of our polynomial operations reveals their essential nature more clearly. Among other things, it finally justifies an assumption we have been making throughout, that $A(x)$ is uniquely characterized by its values at any n points—in fact, we now have an explicit formula that will give us the coefficients of $A(x)$ in this situation. Vandermonde matrices also have the distinction of being quicker to invert than more general matrices, in $O(n^2)$ time instead of $O(n^3)$. However, using this for interpolation would still not be fast enough for us, so once again we turn to our special choice of points—the complex roots of unity.

Interpolation resolved

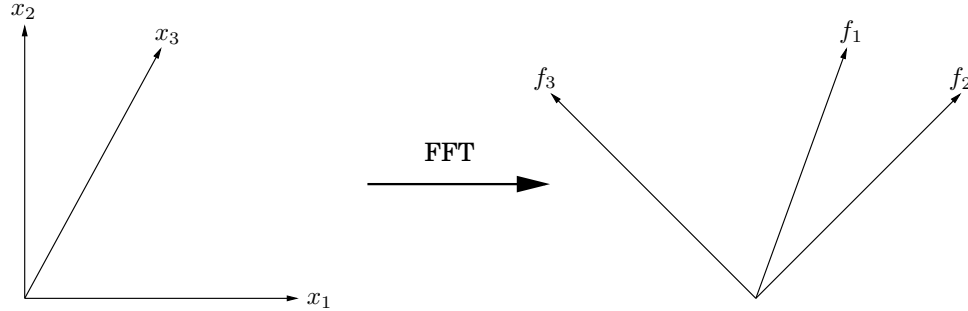
In linear algebra terms, the FFT multiplies an arbitrary n -dimensional vector—which we have been calling the *coefficient representation*—by the $n \times n$ matrix

$$M_n(\omega) = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^j & \omega^{2j} & \cdots & \omega^{(n-1)j} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{(n-1)} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{array}{l} \longleftarrow \text{row for } \omega^0 = 1 \\ \longleftarrow \omega \\ \longleftarrow \omega^2 \\ \vdots \\ \longleftarrow \omega^j \\ \vdots \\ \longleftarrow \omega^{n-1} \end{array}$$

where ω is a complex n th root of unity, and n is a power of 2. Notice how simple this matrix is to describe: its (j, k) th entry (starting row- and column-count at zero) is ω^{jk} .

Multiplication by $M = M_n(\omega)$ maps the k th coordinate axis (the vector with all zeros except for a 1 at position k) onto the k th column of M . Now here's the crucial observation, which we'll prove shortly: *the columns of M are orthogonal (at right angles) to each other.* Therefore they can be thought of as the axes of an alternative coordinate system, which is often called

Figure 2.8 The FFT takes points in the standard coordinate system, whose axes are shown here as x_1, x_2, x_3 , and rotates them into the Fourier basis, whose axes are the columns of $M_n(\omega)$, shown here as f_1, f_2, f_3 . For instance, points in direction x_1 get mapped into direction f_1 .



the *Fourier basis*. The effect of multiplying a vector by M is to rotate it from the standard basis, with the usual set of axes, into the Fourier basis, which is defined by the columns of M (Figure 2.8). The FFT is thus a change of basis, a *rigid rotation*. The inverse of M is the opposite rotation, from the Fourier basis back into the standard basis. When we write out the orthogonality condition precisely, we will be able to read off this inverse transformation with ease:

Inversion formula $M_n(\omega)^{-1} = \frac{1}{n}M_n(\omega^{-1})$.

But ω^{-1} is also an n th root of unity, and so interpolation—or equivalently, multiplication by $M_n(\omega)^{-1}$ —is itself just an FFT operation, but with ω replaced by ω^{-1} .

Now let's get into the details. Take ω to be $e^{2\pi i/n}$ for convenience, and think of the columns of M as vectors in $\mathbb{C}^n$. Recall that the *angle* between two vectors $u = (u_0, \dots, u_{n-1})$ and $v = (v_0, \dots, v_{n-1})$ in $\mathbb{C}^n$ is just a scaling factor times their *inner product*

$$u \cdot v^* = u_0 v_0^* + u_1 v_1^* + \dots + u_{n-1} v_{n-1}^*,$$

where z^* denotes the complex conjugate⁴ of z . This quantity is maximized when the vectors lie in the same direction and is zero when the vectors are orthogonal to each other.

The fundamental observation we need is the following.

Lemma *The columns of matrix M are orthogonal to each other.*

Proof. Take the inner product of any columns j and k of matrix M ,

$$1 + \omega^{j-k} + \omega^{2(j-k)} + \dots + \omega^{(n-1)(j-k)}.$$

This is a geometric series with first term 1, last term $\omega^{(n-1)(j-k)}$, and ratio $\omega^{(j-k)}$. Therefore it evaluates to $(1 - \omega^{n(j-k)})/(1 - \omega^{(j-k)})$, which is 0—except when $j = k$, in which case all terms are 1 and the sum is n . ■

⁴The *complex conjugate* of a complex number $z = re^{i\theta}$ is $z^* = re^{-i\theta}$. The complex conjugate of a vector (or matrix) is obtained by taking the complex conjugates of all its entries.

The orthogonality property can be summarized in the single equation

$$MM^* = nI,$$

since $(MM^*)_{ij}$ is the inner product of the i th and j th columns of M (do you see why?). This immediately implies $M^{-1} = (1/n)M^*$: we have an inversion formula! But is it the same formula we earlier claimed? Let's see—the (j, k) th entry of M^* is the complex conjugate of the corresponding entry of M , in other words ω^{-jk} . Whereupon $M^* = M_n(\omega^{-1})$, and we're done.

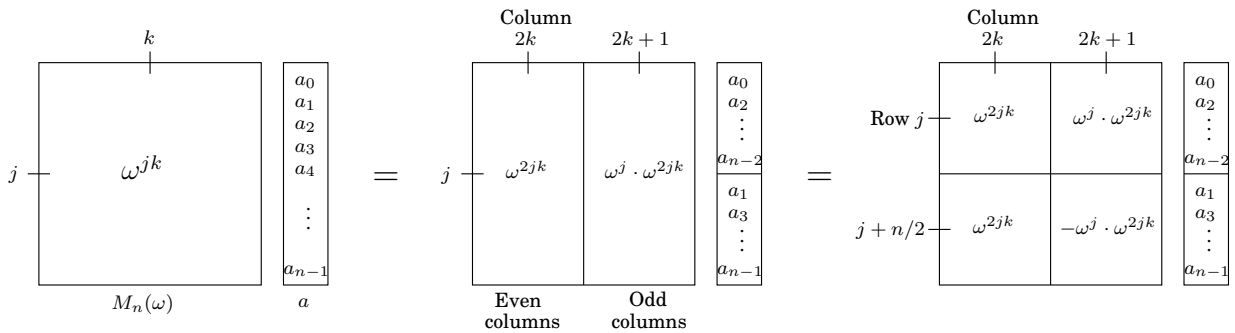
And now we can finally step back and view the whole affair geometrically. The task we need to perform, polynomial multiplication, is a lot easier in the Fourier basis than in the standard basis. Therefore, we first rotate vectors into the Fourier basis (*evaluation*), then perform the task (*multiplication*), and finally rotate back (*interpolation*). The initial vectors are *coefficient representations*, while their rotated counterparts are *value representations*. To efficiently switch between these, back and forth, is the province of the FFT.

2.6.4 A closer look at the fast Fourier transform

Now that our efficient scheme for polynomial multiplication is fully realized, let's hone in more closely on the core subroutine that makes it all possible, the fast Fourier transform.

The definitive FFT algorithm

The FFT takes as input a vector $a = (a_0, \dots, a_{n-1})$ and a complex number ω whose powers $1, \omega, \omega^2, \dots, \omega^{n-1}$ are the complex n th roots of unity. It multiplies vector a by the $n \times n$ matrix $M_n(\omega)$, which has (j, k) th entry (starting row- and column-count at zero) ω^{jk} . The potential for using divide-and-conquer in this matrix-vector multiplication becomes apparent when M 's columns are segregated into evens and odds:



In the second step, we have simplified entries in the bottom half of the matrix using $\omega^{n/2} = -1$ and $\omega^n = 1$. Notice that the top left $n/2 \times n/2$ submatrix is $M_{n/2}(\omega^2)$, as is the one on the bottom left. And the top and bottom right submatrices are almost the same as $M_{n/2}(\omega^2)$, but with their j th rows multiplied through by ω^j and $-\omega^j$, respectively. Therefore the final product is the vector

Figure 2.9 The fast Fourier transform

function FFT(a, ω)

Input: An array $a = (a_0, a_1, \dots, a_{n-1})$, for n a power of 2

A primitive n th root of unity, ω

Output: $M_n(\omega) a$

if $\omega = 1$: **return** a

$(s_0, s_1, \dots, s_{n/2-1}) = \text{FFT}((a_0, a_2, \dots, a_{n-2}), \omega^2)$

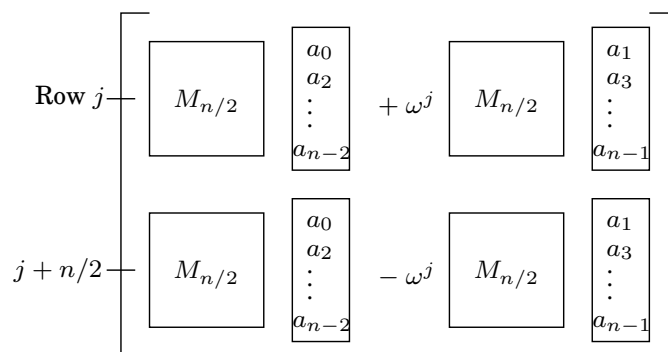
$(s'_0, s'_1, \dots, s'_{n/2-1}) = \text{FFT}((a_1, a_3, \dots, a_{n-1}), \omega^2)$

for $j = 0$ **to** $n/2 - 1$:

$r_j = s_j + \omega^j s'_j$

$r_{j+n/2} = s_j - \omega^j s'_j$

return $(r_0, r_1, \dots, r_{n-1})$



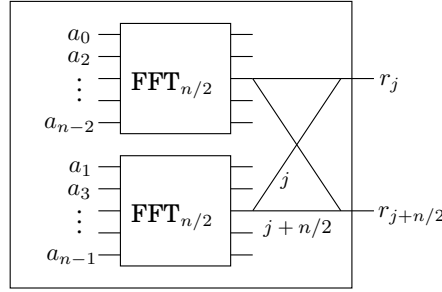
In short, the product of $M_n(\omega)$ with vector $(a_0, \dots, a_{n-1})$, a size- n problem, can be expressed in terms of two size- $n/2$ problems: the product of $M_{n/2}(\omega^2)$ with $(a_0, a_2, \dots, a_{n-2})$ and with $(a_1, a_3, \dots, a_{n-1})$. This divide-and-conquer strategy leads to the definitive FFT algorithm of Figure 2.9, whose running time is $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

The fast Fourier transform unraveled

Throughout all our discussions so far, the fast Fourier transform has remained tightly cooed within a divide-and-conquer formalism. To fully expose its structure, we now unravel the recursion.

The divide-and-conquer step of the FFT can be drawn as a very simple circuit. Here is how a problem of size n is reduced to two subproblems of size $n/2$ (for clarity, one pair of outputs $(j, j + n/2)$ is singled out):

FFT_n (input: $a_0, \dots, a_{n-1}$, output: $r_0, \dots, r_{n-1}$)



We're using a particular shorthand: the edges are wires carrying complex numbers from left to right. A weight of j means "multiply the number on this wire by ω^j ." And when two wires come into a junction from the left, the numbers they are carrying get added up. So the two outputs depicted are executing the commands

$$\begin{aligned} r_j &= s_j + \omega^j s'_j \\ r_{j+n/2} &= s_j - \omega^j s'_j \end{aligned}$$

from the FFT algorithm (Figure 2.9), via a pattern of wires known as a *butterfly*: .

Unraveling the FFT circuit completely for $n = 8$ elements, we get Figure 10.4. Notice the following.

1. For n inputs there are $\log_2 n$ levels, each with n nodes, for a total of $n \log n$ operations.
2. The inputs are arranged in a peculiar order: 0, 4, 2, 6, 1, 5, 3, 7.

Why? Recall that at the top level of recursion, we first bring up the even coefficients of the input and then move on to the odd ones. Then at the next level, the even coefficients of this first group (which therefore are multiples of 4, or equivalently, have zero as their two least significant bits) are brought up, and so on. To put it otherwise, the inputs are arranged by increasing *last* bit of the binary representation of their index, resolving ties by looking at the next more significant bit(s). The resulting order in binary, 000, 100, 010, 110, 001, 101, 011, 111, is the same as the natural one, 000, 001, 010, 011, 100, 101, 110, 111 *except the bits are mirrored!*

3. There is a unique path between each input a_j and each output $A(\omega^k)$.

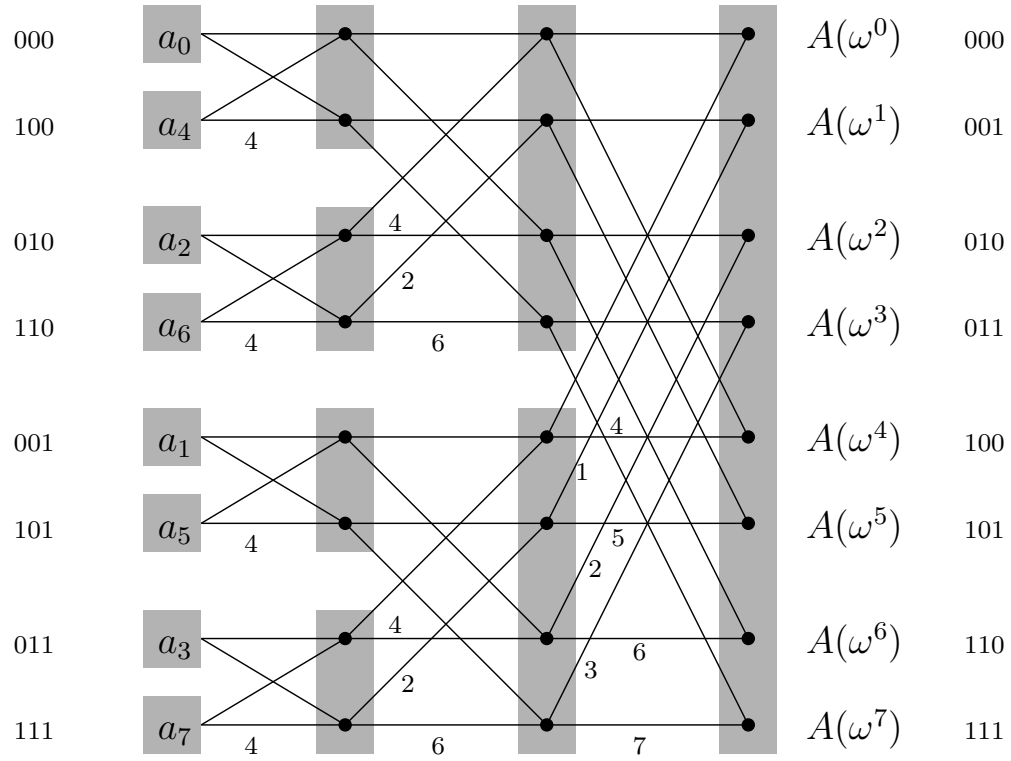
This path is most easily described using the binary representations of j and k (shown in Figure 10.4 for convenience). There are two edges out of each node, one going up (the 0-edge) and one going down (the 1-edge). To get to $A(\omega^k)$ from any input node, simply follow the edges specified in the bit representation of k , starting from the rightmost bit. (Can you similarly specify the path in the reverse direction?)

4. On the path between a_j and $A(\omega^k)$, the labels add up to $jk \bmod 8$.

Since $\omega^8 = 1$, this means that the contribution of input a_j to output $A(\omega^k)$ is $a_j \omega^{jk}$, and therefore the circuit computes correctly the values of polynomial $A(x)$.

5. And finally, notice that the FFT circuit is a natural for parallel computation and direct implementation in hardware.

Figure 2.10 The fast Fourier transform circuit.



The slow spread of a fast algorithm

In 1963, during a meeting of President Kennedy's scientific advisors, John Tukey, a mathematician from Princeton, explained to IBM's Dick Garwin a fast method for computing Fourier transforms. Garwin listened carefully, because he was at the time working on ways to detect nuclear explosions from seismographic data, and Fourier transforms were the bottleneck of his method. When he went back to IBM, he asked John Cooley to implement Tukey's algorithm; they decided that a paper should be published so that the idea could not be patented.

Tukey was not very keen to write a paper on the subject, so Cooley took the initiative. And this is how one of the most famous and most cited scientific papers was published in 1965, co-authored by Cooley and Tukey. The reason Tukey was reluctant to publish the FFT was not secretiveness or pursuit of profit via patents. He just felt that this was a simple observation that was probably already known. This was typical of the period: back then (and for some time later) algorithms were considered second-class mathematical objects, devoid of depth and elegance, and unworthy of serious attention.

But Tukey was right about one thing: it was later discovered that British engineers had used the FFT for hand calculations during the late 1930s. And—to end this chapter with the same great mathematician who started it—a paper by Gauss in the early 1800s on (what else?) interpolation contained essentially the same idea in it! Gauss's paper had remained a secret for so long because it was protected by an old-fashioned cryptographic technique: like most scientific papers of its era, it was written in Latin.

Chapter 5

Divide and Conquer

Divide and conquer refers to a class of algorithmic techniques in which one breaks the input into several parts, solves the problem in each part recursively, and then combines the solutions to these subproblems into an overall solution. In many cases, it can be a simple and powerful method.

Analyzing the running time of a divide and conquer algorithm generally involves solving a *recurrence relation* that bounds the running time recursively in terms of the running time on smaller instances. We begin the chapter with a general discussion of recurrence relations, illustrating how they arise in the analysis and describing methods for working out upper bounds from them.

We then illustrate the use of divide and conquer with applications to a number of different domains: computing a distance function on different rankings of a set of objects; finding the closest pair of points in the plane; multiplying two integers; and smoothing a noisy signal. Divide and conquer will also come up in subsequent chapters, since it is a method that often works well when combined with other algorithm design techniques. For example, in Chapter 6 we will see it combined with dynamic programming to produce a space-efficient solution to a fundamental sequence comparison problem, and in Chapter 13 we will see it combined with randomization to yield a simple and efficient algorithm for computing the median of a set of numbers.

One thing to note about many settings in which divide and conquer is applied, including these, is that the natural brute-force algorithm may already be polynomial time, and the divide and conquer strategy is serving to reduce the running time to a lower polynomial. This is in contrast to most of the problems in the previous chapters, for example, where brute force was exponential and the goal in designing a more sophisticated algorithm was to achieve *any* kind of polynomial running time. For example, we discussed in

Chapter 2 that the natural brute-force algorithm for finding the closest pair among n points in the plane would simply measure all $\Theta(n^2)$ distances, for a (polynomial) running time of $\Theta(n^2)$. Using divide and conquer, we will improve the running time to $O(n \log n)$. At a high level, then, the overall theme of this chapter is the same as what we've been seeing earlier: that improving on brute-force search is a fundamental conceptual hurdle in solving a problem efficiently, and the design of sophisticated algorithms can achieve this. The difference is simply that the distinction between brute-force search and an improved solution here will not always be the distinction between exponential and polynomial.

5.1 A First Recurrence: The Mergesort Algorithm

To motivate the general approach to analyzing divide-and-conquer algorithms, we begin with the *Mergesort* Algorithm. We discussed the Mergesort Algorithm briefly in Chapter 2, when we surveyed common running times for algorithms. Mergesort sorts a given list of numbers by first dividing them into two equal halves, sorting each half separately by recursion, and then combining the results of these recursive calls—in the form of the two sorted halves—using the linear-time algorithm for merging sorted lists that we saw in Chapter 2.

To analyze the running time of Mergesort, we will abstract its behavior into the following template, which describes many common divide-and-conquer algorithms.

(†) Divide the input into two pieces of equal size; solve the two subproblems on these pieces separately by recursion; and then combine the two results into an overall solution, spending only linear time for the initial division and final recombining.

In Mergesort, as in any algorithm that fits this style, we also need a base case for the recursion, typically having it “bottom out” on inputs of some constant size. In the case of Mergesort, we will assume that once the input has been reduced to size 2, we stop the recursion and sort the two elements by simply comparing them to each other.

Consider any algorithm that fits the pattern in (†), and let $T(n)$ denote its worst-case running time on input instances of size n . Supposing that n is even, the algorithm spends $O(n)$ time to divide the input into two pieces of size $n/2$ each; it then spends time $T(n/2)$ to solve each one (since $T(n/2)$ is the worst-case running time for an input of size $n/2$); and finally it spends $O(n)$ time to combine the solutions from the two recursive calls. Thus the running time $T(n)$ satisfies the following *recurrence relation*.

(5.1) For some constant c ,

$$T(n) \leq 2T(n/2) + cn$$

when $n > 2$, and

$$T(2) \leq c.$$

The structure of (5.1) is typical of what recurrences will look like: there's an inequality or equation that bounds $T(n)$ in terms of an expression involving $T(k)$ for smaller values k ; and there is a base case that generally says that $T(n)$ is equal to a constant when n is a constant. Note that one can also write (5.1) more informally as $T(n) \leq 2T(n/2) + O(n)$, suppressing the constant c . However, it is generally useful to make c explicit when analyzing the recurrence.

To keep the exposition simpler, we will generally assume that parameters like n are even when needed. This is somewhat imprecise usage; without this assumption, the two recursive calls would be on problems of size $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$, and the recurrence relation would say that

$$T(n) \leq T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + cn$$

for $n \geq 2$. Nevertheless, for all the recurrences we consider here (and for most that arise in practice), the asymptotic bounds are not affected by the decision to ignore all the floors and ceilings, and it makes the symbolic manipulation much cleaner.

Now (5.1) does not explicitly provide an asymptotic bound on the growth rate of the function T ; rather, it specifies $T(n)$ implicitly in terms of its values on smaller inputs. To obtain an explicit bound, we need to solve the recurrence relation so that T appears only on the left-hand side of the inequality, not the right-hand side as well.

Recurrence solving is a task that has been incorporated into a number of standard computer algebra systems, and the solution to many standard recurrences can now be found by automated means. It is still useful, however, to understand the process of solving recurrences and to recognize which recurrences lead to good running times, since the design of an efficient divide-and-conquer algorithm is heavily intertwined with an understanding of how a recurrence relation determines a running time.

Approaches to Solving Recurrences

There are two basic ways one can go about solving a recurrence, each of which we describe in more detail below.

- The most intuitively natural way to search for a solution to a recurrence is to “unroll” the recursion, accounting for the running time across the first few levels, and identify a pattern that can be continued as the recursion expands. One then sums the running times over all levels of the recursion (i.e., until it “bottoms out” on subproblems of constant size) and thereby arrives at a total running time.
- A second way is to start with a guess for the solution, substitute it into the recurrence relation, and check that it works. Formally, one justifies this plugging-in using an argument by induction on n . There is a useful variant of this method in which one has a general form for the solution, but does not have exact values for all the parameters. By leaving these parameters unspecified in the substitution, one can often work them out as needed.

We now discuss each of these approaches, using the recurrence in (5.1) as an example.

Unrolling the Mergesort Recurrence

Let’s start with the first approach to solving the recurrence in (5.1). The basic argument is depicted in Figure 5.1.

- *Analyzing the first few levels:* At the first level of recursion, we have a single problem of size n , which takes time at most cn plus the time spent in all subsequent recursive calls. At the next level, we have two problems each of size $n/2$. Each of these takes time at most $cn/2$, for a total of at most cn , again plus the time in subsequent recursive calls. At the third level, we have four problems each of size $n/4$, each taking time at most $cn/4$, for a total of at most cn .

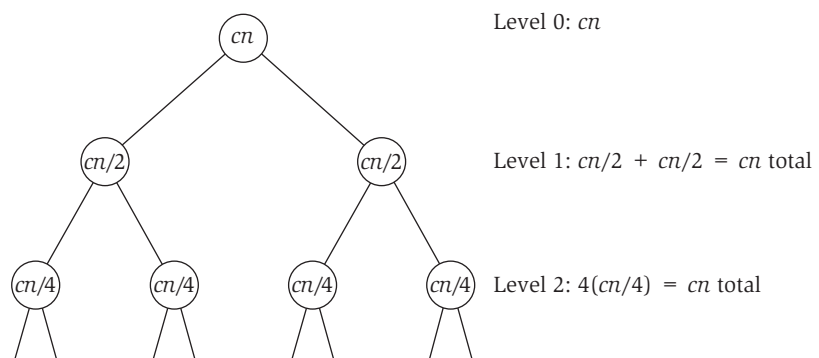


Figure 5.1 Unrolling the recurrence $T(n) \leq 2T(n/2) + O(n)$.

- *Identifying a pattern:* What's going on in general? At level j of the recursion, the number of subproblems has doubled j times, so there are now a total of 2^j . Each has correspondingly shrunk in size by a factor of two j times, and so each has size $n/2^j$, and hence each takes time at most $cn/2^j$. Thus level j contributes a total of at most $2^j(cn/2^j) = cn$ to the total running time.
- *Summing over all levels of recursion:* We've found that the recurrence in (5.1) has the property that the same upper bound of cn applies to total amount of work performed at each level. The number of times the input must be halved in order to reduce its size from n to 2 is $\log_2 n$. So summing the cn work over $\log n$ levels of recursion, we get a total running time of $O(n \log n)$.

We summarize this in the following claim.

(5.2) Any function $T(\cdot)$ satisfying (5.1) is bounded by $O(n \log n)$, when $n > 1$.

Substituting a Solution into the Mergesort Recurrence

The argument establishing (5.2) can be used to determine that the function $T(n)$ is bounded by $O(n \log n)$. If, on the other hand, we have a guess for the running time that we want to verify, we can do so by plugging it into the recurrence as follows.

Suppose we believe that $T(n) \leq cn \log_2 n$ for all $n \geq 2$, and we want to check whether this is indeed true. This clearly holds for $n = 2$, since in this case $cn \log_2 n = 2c$, and (5.1) explicitly tells us that $T(2) \leq c$. Now suppose, by induction, that $T(m) \leq cm \log_2 m$ for all values of m less than n , and we want to establish this for $T(n)$. We do this by writing the recurrence for $T(n)$ and plugging in the inequality $T(n/2) \leq c(n/2) \log_2(n/2)$. We then simplify the resulting expression by noticing that $\log_2(n/2) = (\log_2 n) - 1$. Here is the full calculation.

$$\begin{aligned}
 T(n) &\leq 2T(n/2) + cn \\
 &\leq 2c(n/2) \log_2(n/2) + cn \\
 &= cn[(\log_2 n) - 1] + cn \\
 &= (cn \log_2 n) - cn + cn \\
 &= cn \log_2 n.
 \end{aligned}$$

This establishes the bound we want for $T(n)$, assuming it holds for smaller values $m < n$, and thus it completes the induction argument.

An Approach Using Partial Substitution

There is a somewhat weaker kind of substitution one can do, in which one guesses the overall form of the solution without pinning down the exact values of all the constants and other parameters at the outset.

Specifically, suppose we believe that $T(n) = O(n \log n)$, but we're not sure of the constant inside the $O(\cdot)$ notation. We can use the substitution method even without being sure of this constant, as follows. We first write $T(n) \leq kn \log_b n$ for some constant k and base b that we'll determine later. (Actually, the base and the constant we'll end up needing are related to each other, since we saw in Chapter 2 that one can change the base of the logarithm by simply changing the multiplicative constant in front.)

Now we'd like to know whether there is any choice of k and b that will work in an inductive argument. So we try out one level of the induction as follows.

$$T(n) \leq 2T(n/2) + cn \leq 2k(n/2) \log_b(n/2) + cn.$$

It's now very tempting to choose the base $b = 2$ for the logarithm, since we see that this will let us apply the simplification $\log_2(n/2) = (\log_2 n) - 1$. Proceeding with this choice, we have

$$\begin{aligned} T(n) &\leq 2k(n/2) \log_2(n/2) + cn \\ &= 2k(n/2)[(\log_2 n) - 1] + cn \\ &= kn[(\log_2 n) - 1] + cn \\ &= (kn \log_2 n) - kn + cn. \end{aligned}$$

Finally, we ask: Is there a choice of k that will cause this last expression to be bounded by $kn \log_2 n$? The answer is clearly yes; we just need to choose any k that is at least as large as c , and we get

$$T(n) \leq (kn \log_2 n) - kn + cn \leq kn \log_2 n,$$

which completes the induction.

Thus the substitution method can actually be useful in working out the exact constants when one has some guess of the general form of the solution.

5.2 Further Recurrence Relations

We've just worked out the solution to a recurrence relation, (5.1), that will come up in the design of several divide-and-conquer algorithms later in this chapter. As a way to explore this issue further, we now consider a class of recurrence relations that generalizes (5.1), and show how to solve the recurrences in this class. Other members of this class will arise in the design of algorithms both in this and in later chapters.

This more general class of algorithms is obtained by considering divide-and-conquer algorithms that create recursive calls on q subproblems of size $n/2$ each and then combine the results in $O(n)$ time. This corresponds to the Mergesort recurrence (5.1) when $q = 2$ recursive calls are used, but other algorithms find it useful to spawn $q > 2$ recursive calls, or just a single ($q = 1$) recursive call. In fact, we will see the case $q > 2$ later in this chapter when we design algorithms for integer multiplication; and we will see a variant on the case $q = 1$ much later in the book, when we design a randomized algorithm for median finding in Chapter 13.

If $T(n)$ denotes the running time of an algorithm designed in this style, then $T(n)$ obeys the following recurrence relation, which directly generalizes (5.1) by replacing 2 with q :

(5.3) For some constant c ,

$$T(n) \leq qT(n/2) + cn$$

when $n > 2$, and

$$T(2) \leq c.$$

We now describe how to solve (5.3) by the methods we've seen above: unrolling, substitution, and partial substitution. We treat the cases $q > 2$ and $q = 1$ separately, since they are qualitatively different from each other—and different from the case $q = 2$ as well.

The Case of $q > 2$ Subproblems

We begin by unrolling (5.3) in the case $q > 2$, following the style we used earlier for (5.1). We will see that the punch line ends up being quite different.

- *Analyzing the first few levels:* We show an example of this for the case $q = 3$ in Figure 5.2. At the first level of recursion, we have a single problem of size n , which takes time at most cn plus the time spent in all subsequent recursive calls. At the next level, we have q problems, each of size $n/2$. Each of these takes time at most $cn/2$, for a total of at most $(q/2)cn$, again plus the time in subsequent recursive calls. The next level yields q^2 problems of size $n/4$ each, for a total time of $(q^2/4)cn$. Since $q > 2$, we see that the total work per level is *increasing* as we proceed through the recursion.
- *Identifying a pattern:* At an arbitrary level j , we have q^j distinct instances, each of size $n/2^j$. Thus the total work performed at level j is $q^j(cn/2^j) = (q/2)^j cn$.

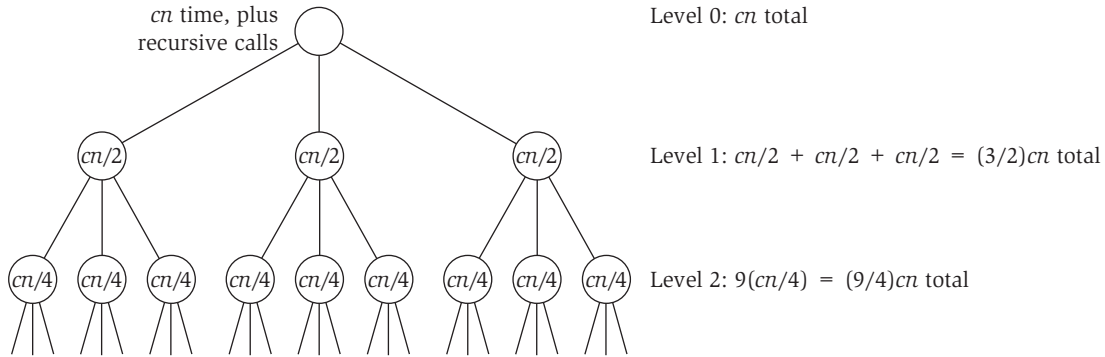


Figure 5.2 Unrolling the recurrence $T(n) \leq 3T(n/2) + O(n)$.

- *Summing over all levels of recursion:* As before, there are $\log_2 n$ levels of recursion, and the total amount of work performed is the sum over all these:

$$T(n) \leq \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j cn = cn \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j.$$

This is a geometric sum, consisting of powers of $r = q/2$. We can use the formula for a geometric sum when $r > 1$, which gives us the formula

$$T(n) \leq cn \left(\frac{r^{\log_2 n} - 1}{r - 1} \right) \leq cn \left(\frac{r^{\log_2 n}}{r - 1} \right).$$

Since we're aiming for an asymptotic upper bound, it is useful to figure out what's simply a constant; we can pull out the factor of $r - 1$ from the denominator, and write the last expression as

$$T(n) \leq \left(\frac{c}{r - 1} \right) nr^{\log_2 n}.$$

Finally, we need to figure out what $r^{\log_2 n}$ is. Here we use a very handy identity, which says that, for any $a > 1$ and $b > 1$, we have $a^{\log b} = b^{\log a}$. Thus

$$r^{\log_2 n} = n^{\log_2 r} = n^{\log_2(q/2)} = n^{(\log_2 q) - 1}.$$

Thus we have

$$T(n) \leq \left(\frac{c}{r - 1} \right) n \cdot n^{(\log_2 q) - 1} \leq \left(\frac{c}{r - 1} \right) n^{\log_2 q} = O(n^{\log_2 q}).$$

We sum this up as follows.

(5.4) Any function $T(\cdot)$ satisfying (5.3) with $q > 2$ is bounded by $O(n^{\log_2 q})$.

So we find that the running time is more than linear, since $\log_2 q > 1$, but still polynomial in n . Plugging in specific values of q , the running time is $O(n^{\log_2 3}) = O(n^{1.59})$ when $q = 3$; and the running time is $O(n^{\log_2 4}) = O(n^2)$ when $q = 4$. This increase in running time as q increases makes sense, of course, since the recursive calls generate more work for larger values of q .

Applying Partial Substitution The appearance of $\log_2 q$ in the exponent followed naturally from our solution to (5.3), but it's not necessarily an expression one would have guessed at the outset. We now consider how an approach based on partial substitution into the recurrence yields a different way of discovering this exponent.

Suppose we guess that the solution to (5.3), when $q > 2$, has the form $T(n) \leq kn^d$ for some constants $k > 0$ and $d > 1$. This is quite a general guess, since we haven't even tried specifying the exponent d of the polynomial. Now let's try starting the inductive argument and seeing what constraints we need on k and d . We have

$$T(n) \leq qT(n/2) + cn,$$

and applying the inductive hypothesis to $T(n/2)$, this expands to

$$\begin{aligned} T(n) &\leq qk \left(\frac{n}{2}\right)^d + cn \\ &= \frac{q}{2^d} kn^d + cn. \end{aligned}$$

This is remarkably close to something that works: if we choose d so that $q/2^d = 1$, then we have $T(n) \leq kn^d + cn$, which is almost right except for the extra term cn . So let's deal with these two issues: first, how to choose d so we get $q/2^d = 1$; and second, how to get rid of the cn term.

Choosing d is easy: we want $2^d = q$, and so $d = \log_2 q$. Thus we see that the exponent $\log_2 q$ appears very naturally once we decide to discover which value of d works when substituted into the recurrence.

But we still have to get rid of the cn term. To do this, we change the form of our guess for $T(n)$ so as to explicitly subtract it off. Suppose we try the form $T(n) \leq kn^d - \ell n$, where we've now decided that $d = \log_2 q$ but we haven't fixed the constants k or ℓ . Applying the new formula to $T(n/2)$, this expands to

$$\begin{aligned}
T(n) &\leq qk \left(\frac{n}{2}\right)^d - q\ell \left(\frac{n}{2}\right) + cn \\
&= \frac{q}{2^d} kn^d - \frac{q\ell}{2} n + cn \\
&= kn^d - \frac{q\ell}{2} n + cn \\
&= kn^d - \left(\frac{q\ell}{2} - c\right)n.
\end{aligned}$$

This now works completely, if we simply choose ℓ so that $(\frac{q\ell}{2} - c) = \ell$: in other words, $\ell = 2c/(q - 2)$. This completes the inductive step for n . We also need to handle the base case $n = 2$, and this we do using the fact that the value of k has not yet been fixed: we choose k large enough so that the formula is a valid upper bound for the case $n = 2$.

The Case of One Subproblem

We now consider the case of $q = 1$ in (5.3), since this illustrates an outcome of yet another flavor. While we won't see a direct application of the recurrence for $q = 1$ in this chapter, a variation on it comes up in Chapter 13, as we mentioned earlier.

We begin by unrolling the recurrence to try constructing a solution.

- *Analyzing the first few levels:* We show the first few levels of the recursion in Figure 5.3. At the first level of recursion, we have a single problem of size n , which takes time at most cn plus the time spent in all subsequent recursive calls. The next level has one problem of size $n/2$, which contributes $cn/2$, and the level after that has one problem of size $n/4$, which contributes $cn/4$. So we see that, unlike the previous case, the total work per level when $q = 1$ is actually *decreasing* as we proceed through the recursion.
- *Identifying a pattern:* At an arbitrary level j , we still have just one instance; it has size $n/2^j$ and contributes $cn/2^j$ to the running time.
- *Summing over all levels of recursion:* There are $\log_2 n$ levels of recursion, and the total amount of work performed is the sum over all these:

$$T(n) \leq \sum_{j=0}^{\log_2 n - 1} \frac{cn}{2^j} = cn \sum_{j=0}^{\log_2 n - 1} \left(\frac{1}{2^j}\right).$$

This geometric sum is very easy to work out; even if we continued it to infinity, it would converge to 2. Thus we have

$$T(n) \leq 2cn = O(n).$$

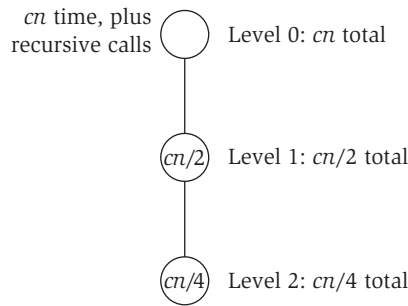


Figure 5.3 Unrolling the recurrence $T(n) \leq T(n/2) + O(n)$.

We sum this up as follows.

(5.5) Any function $T(\cdot)$ satisfying (5.3) with $q = 1$ is bounded by $O(n)$.

This is counterintuitive when you first see it. The algorithm is performing $\log n$ levels of recursion, but the overall running time is still linear in n . The point is that a geometric series with a decaying exponent is a powerful thing: fully half the work performed by the algorithm is being done at the top level of the recursion.

It is also useful to see how partial substitution into the recurrence works very well in this case. Suppose we guess, as before, that the form of the solution is $T(n) \leq kn^d$. We now try to establish this by induction using (5.3), assuming that the solution holds for the smaller value $n/2$:

$$\begin{aligned} T(n) &\leq T(n/2) + cn \\ &\leq k \left(\frac{n}{2}\right)^d + cn \\ &= \frac{k}{2^d} n^d + cn. \end{aligned}$$

If we now simply choose $d = 1$ and $k = 2c$, we have

$$T(n) \leq \frac{k}{2} n + cn = \left(\frac{k}{2} + c\right)n = kn,$$

which completes the induction.

The Effect of the Parameter q . It is worth reflecting briefly on the role of the parameter q in the class of recurrences $T(n) \leq qT(n/2) + O(n)$ defined by (5.3). When $q = 1$, the resulting running time is linear; when $q = 2$, it's $O(n \log n)$; and when $q > 2$, it's a polynomial bound with an exponent larger than 1 that grows with q . The reason for this range of different running times lies in where

most of the work is spent in the recursion: when $q = 1$, the total running time is dominated by the top level, whereas when $q > 2$ it's dominated by the work done on constant-size subproblems at the bottom of the recursion. Viewed this way, we can appreciate that the recurrence for $q = 2$ really represents a “knife-edge”—the amount of work done at each level is *exactly the same*, which is what yields the $O(n \log n)$ running time.

A Related Recurrence: $T(n) \leq 2T(n/2) + O(n^2)$

We conclude our discussion with one final recurrence relation; it is illustrative both as another application of a decaying geometric sum and as an interesting contrast with the recurrence (5.1) that characterized Mergesort. Moreover, we will see a close variant of it in Chapter 6, when we analyze a divide-and-conquer algorithm for solving the Sequence Alignment Problem using a small amount of working memory.

The recurrence is based on the following divide-and-conquer structure.

Divide the input into two pieces of equal size; solve the two subproblems on these pieces separately by recursion; and then combine the two results into an overall solution, spending quadratic time for the initial division and final recombining.

For our purposes here, we note that this style of algorithm has a running time $T(n)$ that satisfies the following recurrence.

(5.6) For some constant c ,

$$T(n) \leq 2T(n/2) + cn^2$$

when $n > 2$, and

$$T(2) \leq c.$$

One's first reaction is to guess that the solution will be $T(n) = O(n^2 \log n)$, since it looks almost identical to (5.1) except that the amount of work per level is larger by a factor equal to the input size. In fact, this upper bound is correct (it would need a more careful argument than what's in the previous sentence), but it will turn out that we can also show a stronger upper bound.

We'll do this by unrolling the recurrence, following the standard template for doing this.

- *Analyzing the first few levels:* At the first level of recursion, we have a single problem of size n , which takes time at most cn^2 plus the time spent in all subsequent recursive calls. At the next level, we have two problems, each of size $n/2$. Each of these takes time at most $c(n/2)^2 = cn^2/4$, for a

total of at most $cn^2/2$, again plus the time in subsequent recursive calls. At the third level, we have four problems each of size $n/4$, each taking time at most $c(n/4)^2 = cn^2/16$, for a total of at most $cn^2/4$. Already we see that something is different from our solution to the analogous recurrence (5.1); whereas the total amount of work per level remained the same in that case, here it's decreasing.

- *Identifying a pattern:* At an arbitrary level j of the recursion, there are 2^j subproblems, each of size $n/2^j$, and hence the total work at this level is bounded by $2^j c(n/2^j)^2 = cn^2/2^j$.
- *Summing over all levels of recursion:* Having gotten this far in the calculation, we've arrived at almost exactly the same sum that we had for the case $q = 1$ in the previous recurrence. We have

$$T(n) \leq \sum_{j=0}^{\log_2 n-1} \frac{cn^2}{2^j} = cn^2 \sum_{j=0}^{\log_2 n-1} \left(\frac{1}{2}\right)^j \leq 2cn^2 = O(n^2),$$

where the second inequality follows from the fact that we have a convergent geometric sum.

In retrospect, our initial guess of $T(n) = O(n^2 \log n)$, based on the analogy to (5.1), was an overestimate because of how quickly n^2 decreases as we replace it with $(\frac{n}{2})^2$, $(\frac{n}{4})^2$, $(\frac{n}{8})^2$, and so forth in the unrolling of the recurrence. This means that we get a geometric sum, rather than one that grows by a fixed amount over all n levels (as in the solution to (5.1)).

5.3 Counting Inversions

We've spent some time discussing approaches to solving a number of common recurrences. The remainder of the chapter will illustrate the application of divide-and-conquer to problems from a number of different domains; we will use what we've seen in the previous sections to bound the running times of these algorithms. We begin by showing how a variant of the Mergesort technique can be used to solve a problem that is not directly related to sorting numbers.



The Problem

We will consider a problem that arises in the analysis of *rankings*, which are becoming important to a number of current applications. For example, a number of sites on the Web make use of a technique known as *collaborative filtering*, in which they try to match your preferences (for books, movies, restaurants) with those of other people out on the Internet. Once the Web site has identified people with “similar” tastes to yours—based on a comparison

of how you and they rate various things—it can recommend new things that these other people have liked. Another application arises in *meta-search tools* on the Web, which execute the same query on many different search engines and then try to synthesize the results by looking for similarities and differences among the various rankings that the search engines return.

A core issue in applications like this is the problem of comparing two rankings. You rank a set of n movies, and then a collaborative filtering system consults its database to look for other people who had “similar” rankings. But what’s a good way to measure, numerically, how similar two people’s rankings are? Clearly an identical ranking is very similar, and a completely reversed ranking is very different; we want something that interpolates through the middle region.

Let’s consider comparing your ranking and a stranger’s ranking of the same set of n movies. A natural method would be to label the movies from 1 to n according to your ranking, then order these labels according to the stranger’s ranking, and see how many pairs are “out of order.” More concretely, we will consider the following problem. We are given a sequence of n numbers $a_1, \dots, a_n$; we will assume that all the numbers are distinct. We want to define a measure that tells us how far this list is from being in ascending order; the value of the measure should be 0 if $a_1 < a_2 < \dots < a_n$, and should increase as the numbers become more scrambled.

A natural way to quantify this notion is by counting the number of *inversions*. We say that two indices $i < j$ form an inversion if $a_i > a_j$, that is, if the two elements a_i and a_j are “out of order.” We will seek to determine the number of inversions in the sequence $a_1, \dots, a_n$.

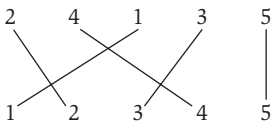


Figure 5.4 Counting the number of inversions in the sequence 2, 4, 1, 3, 5. Each crossing pair of line segments corresponds to one pair that is in the opposite order in the input list and the ascending list—in other words, an inversion.

Just to pin down this definition, consider an example in which the sequence is 2, 4, 1, 3, 5. There are three inversions in this sequence: (2, 1), (4, 1), and (4, 3). There is also an appealing geometric way to visualize the inversions, pictured in Figure 5.4: we draw the sequence of input numbers in the order they’re provided, and below that in ascending order. We then draw a line segment between each number in the top list and its copy in the lower list. Each crossing pair of line segments corresponds to one pair that is in the opposite order in the two lists—in other words, an inversion.

Note how the number of inversions is a measure that smoothly interpolates between complete agreement (when the sequence is in ascending order, then there are no inversions) and complete disagreement (if the sequence is in descending order, then every pair forms an inversion, and so there are $\binom{n}{2}$ of them).

Designing and Analyzing the Algorithm

What is the simplest algorithm to count inversions? Clearly, we could look at every pair of numbers (a_i, a_j) and determine whether they constitute an inversion; this would take $O(n^2)$ time.

We now show how to count the number of inversions much more quickly, in $O(n \log n)$ time. Note that since there can be a quadratic number of inversions, such an algorithm must be able to compute the total number without ever looking at each inversion individually. The basic idea is to follow the strategy $(\dagger)$ defined in Section 5.1. We set $m = \lceil n/2 \rceil$ and divide the list into the two pieces $a_1, \dots, a_m$ and $a_{m+1}, \dots, a_n$. We first count the number of inversions in each of these two halves separately. Then we count the number of inversions (a_i, a_j) , where the two numbers belong to different halves; the trick is that we must do this part in $O(n)$ time, if we want to apply (5.2). Note that these first-half/second-half inversions have a particularly nice form: they are precisely the pairs (a_i, a_j) , where a_i is in the first half, a_j is in the second half, and $a_i > a_j$.

To help with counting the number of inversions between the two halves, we will make the algorithm recursively sort the numbers in the two halves as well. Having the recursive step do a bit more work (sorting as well as counting inversions) will make the “combining” portion of the algorithm easier.

So the crucial routine in this process is **Merge-and-Count**. Suppose we have recursively sorted the first and second halves of the list and counted the inversions in each. We now have two sorted lists A and B , containing the first and second halves, respectively. We want to produce a single sorted list C from their union, while also counting the number of pairs (a, b) with $a \in A$, $b \in B$, and $a > b$. By our previous discussion, this is precisely what we will need for the “combining” step that computes the number of first-half/second-half inversions.

This is closely related to the simpler problem we discussed in Chapter 2, which formed the corresponding “combining” step for Mergesort: there we had two sorted lists A and B , and we wanted to merge them into a single sorted list in $O(n)$ time. The difference here is that we want to do something extra: not only should we produce a single sorted list from A and B , but we should also count the number of “inverted pairs” (a, b) where $a \in A$, $b \in B$, and $a > b$.

It turns out that we will be able to do this in very much the same style that we used for merging. Our **Merge-and-Count** routine will walk through the sorted lists A and B , removing elements from the front and appending them to the sorted list C . In a given step, we have a *Current* pointer into each list, showing our current position. Suppose that these pointers are currently

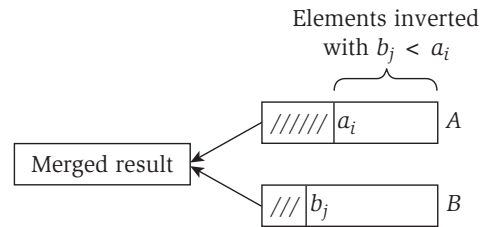


Figure 5.5 Merging two sorted lists while also counting the number of inversions between them.

at elements a_i and b_j . In one step, we compare the elements a_i and b_j being pointed to in each list, remove the smaller one from its list, and append it to the end of list C.

This takes care of merging. How do we also count the number of inversions? Because A and B are sorted, it is actually very easy to keep track of the number of inversions we encounter. Every time the element a_i is appended to C, no new inversions are encountered, since a_i is smaller than everything left in list B, and it comes before all of them. On the other hand, if b_j is appended to list C, then it is smaller than all the remaining items in A, and it comes after all of them, so we increase our count of the number of inversions by the number of elements remaining in A. This is the crucial idea: in constant time, we have accounted for a potentially large number of inversions. See Figure 5.5 for an illustration of this process.

To summarize, we have the following algorithm.

`Merge-and-Count(A,B)`

 Maintain a *Current* pointer into each list, initialized to
 point to the front elements

 Maintain a variable *Count* for the number of inversions,
 initialized to 0

 While both lists are nonempty:

 Let a_i and b_j be the elements pointed to by the *Current* pointer

 Append the smaller of these two to the output list

 If b_j is the smaller element then

 Increment *Count* by the number of elements remaining in A

 Endif

 Advance the *Current* pointer in the list from which the
 smaller element was selected.

 EndWhile